



IPV6-ONLY WIREGUARD VPN WITH NAT66 USING MIKROTIK ROUTEROS AND GNS3

A Practical Implementation of Remote IPv6 Connectivity Using ULA Addressing,
WireGuard, and NAT66



JANUARY 1, 2026

SUBASH SUBEDI

Table of Contents

1.	Introduction	1
1.1.	Project Background.....	1
1.2.	Problem Statement	2
1.3.	Aim and Objectives	3
1.3.1.	Aim.....	3
1.3.2.	Objectives	3
2.	IPv4 CGNAT and IPv6	4
2.1.	IPv4 CGNAT Problem.....	4
2.2.	How IPv6 Helps Remote Access	5
2.3.	Why IPv6 Is Used in This Project.....	6
3.	WireGuard and NAT66 Overview.....	9
3.1.	WireGuard VPN	9
3.2.	IPv6 ULA Addressing.....	10
3.3.	NAT66.....	11
3.4.	Proposed Solution.....	12
4.	Network Topology and Addressing Plan	14
4.1.	GNS3 Topology	14
4.2.	IPv6 Addressing Plan.....	16
4.3.	Traffic Flow	16
5.	IPv6 WAN Configuration	19
5.1.	Configure IPv6 WAN	19
5.2.	Verify Global IPv6 Address	20
5.3.	Verify IPv6 Default Route.....	21
5.4.	Test IPv6 Connectivity	22
6.	WireGuard IPv6 Configuration	24
6.1.	Create WireGuard Interface	24

6.2.	Assign ULA IPv6 Address	25
6.3.	Create WireGuard Peer	26
6.4.	Configure Client	28
6.5.	Generate Client Configuration / QR Code.....	30
7.	IPv6 Firewall Configuration	33
7.1.	Allow WireGuard Traffic	33
7.2.	Allow VPN Client Traffic.....	34
7.3.	Allow IPv6 Forwarding	35
8.	NAT66 Configuration.....	38
8.1.	Why NAT66 Is Required	38
8.2.	Configure NAT66	39
8.3.	Verify NAT66 Traffic.....	40
9.	Testing and Verification.....	43
9.1.	WireGuard Handshake	43
9.2.	IPv6 Ping Test.....	44
9.3.	IPv6 Internet Test.....	45
9.4.	NAT66 Counter Verification	47
9.5.	Packet and Traffic Analysis.....	48
10.	Troubleshooting.....	51
10.1.	WireGuard Endpoint Issues.....	51
10.2.	QR Code Import Issues	52
10.3.	IPv6 Routing Issues	53
10.4.	NAT66 Issues.....	54
11.	Cost and Real-World Use for SMEs	56
11.1.	Public IPv4 vs IPv6.....	56
11.2.	Cost Benefits	57
11.3.	Real-World Deployment Considerations.....	58

12.	Results and Conclusion	60
13.	References	62
14.	Appendix	62
14.1.	Complete MikroTik Configuration	62
14.2.	Verification Command	64

1. Introduction

This project focuses on designing and testing a low-cost remote access VPN solution using IPv6, WireGuard, NAT66, MikroTik RouterOS, and GNS3. The main purpose of this project is to explore how IPv6 can be used as an alternative when users do not have a directly reachable public IPv4 address because of Carrier-Grade NAT (CGNAT).

The whole setup is first created and tested in GNS3 using MikroTik CHR. WireGuard is used to create the VPN tunnel, while Unique Local IPv6 Addresses are used for VPN clients. NAT66 is then used so that the VPN clients can access the IPv6 Internet through the global IPv6 address available on the MikroTik router.

This project mainly looks at whether this type of setup can be useful for small and medium-sized businesses that need secure remote access but do not want to pay extra for a dedicated Internet connection or public IPv4 address.

1.1. Project Background

IPv4 has been used for many years for Internet communication, but the number of available IPv4 addresses is limited. Because of this shortage, many Internet Service Providers now use Carrier-Grade NAT, also known as CGNAT.

With CGNAT, multiple customers share one public IPv4 address. The customer normally receives a private IPv4 address from the ISP instead of a directly reachable public IPv4 address. This works well for normal Internet browsing, but it creates problems when a user wants to host a service or create a VPN server that needs incoming connections from the Internet.

IPv6 was introduced to solve the address shortage problem. IPv6 provides a much larger address space and allows ISPs to provide globally routable IPv6 connectivity without sharing addresses in the same way as IPv4 CGNAT.

Because of this, IPv6 can also be used for services such as remote access VPNs. In this project, the global IPv6 address available on the MikroTik router is used as the WireGuard VPN endpoint.

The WireGuard clients use ULA IPv6 addresses inside the VPN tunnel. Since ULA addresses are not directly routable on the public Internet, NAT66 is used to translate the private IPv6 addresses of the VPN clients to the global IPv6 address of the MikroTik router.

The project is tested in GNS3 before considering how the same design could be used in a real small-business environment.

1.2. Problem Statement

Many home users and small businesses use normal broadband Internet connections that are placed behind IPv4 CGNAT. Because they do not receive a directly reachable public IPv4 address, it becomes difficult to host VPN servers or remotely access internal office resources.

One possible solution is to purchase a public IPv4 address or a dedicated business Internet connection. However, this may increase the monthly Internet cost and may not be practical for small organizations.

At the same time, many ISPs already provide IPv6 connectivity. If a globally routable IPv6 address is available, it may be possible to use that address as the public endpoint of a VPN instead of depending on a public IPv4 address.

Therefore, this project investigates whether IPv6, WireGuard, NAT66, and MikroTik RouterOS can be combined to provide a simple and cost-effective remote access VPN solution.

The implementation will be created and tested in GNS3 so that the configuration, routing, VPN connectivity, NAT66 operation, and security rules can be verified before considering real-world deployment.

1.3. Aim and Objectives

1.3.1. Aim

To design, implement, and test a cost-effective IPv6-based remote access VPN using WireGuard, NAT66, MikroTik RouterOS, and GNS3 as an alternative to depending on a public IPv4 address.

1.3.2. Objectives

The objectives of this project are:

- To understand the limitations of IPv4 CGNAT for remote access.
- To study how IPv6 can be used for globally reachable VPN connectivity.
- To design an IPv6-based WireGuard VPN topology in GNS3.
- To configure IPv6 connectivity on MikroTik RouterOS.
- To create a WireGuard VPN using IPv6 as the public endpoint.
- To use ULA IPv6 addresses for WireGuard VPN clients.
- To configure NAT66 for VPN client Internet access.
- To configure IPv6 firewall rules for the WireGuard VPN.
- To verify WireGuard handshakes and client connectivity.
- To test IPv6 routing and NAT66 operation.
- To troubleshoot common WireGuard and IPv6 configuration problems.
- To evaluate whether the proposed solution can be suitable for small and medium-sized businesses with limited budgets.

2. IPv4 CGNAT and IPv6

IPv4 and IPv6 are both used by Internet Service Providers to provide Internet connectivity. However, they work differently when it comes to public addressing and remote access.

Due to the shortage of IPv4 addresses, many ISPs use Carrier-Grade NAT (CGNAT), where several customers share a single public IPv4 address. This is suitable for normal Internet use, but it creates problems when users want to host services or establish inbound VPN connections.

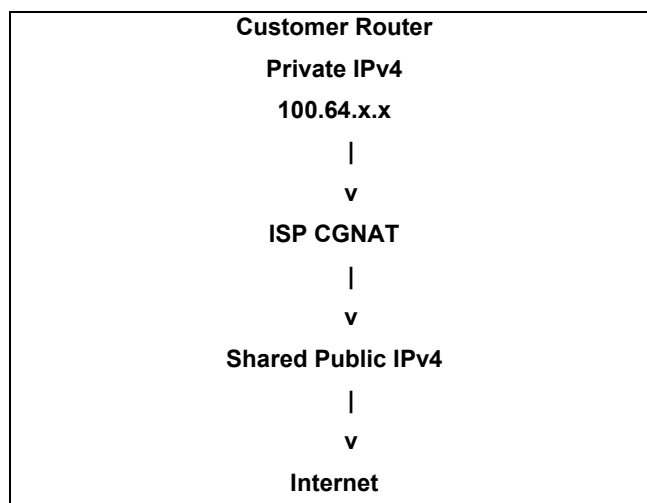
IPv6 provides a much larger address space and allows customers to receive globally routable IPv6 connectivity. Because of this, IPv6 can be useful for remote access solutions such as WireGuard VPN.

2.1. IPv4 CGNAT Problem

IPv4 has a limited address space, and the number of available public IPv4 addresses is not enough for every Internet user and device. To solve this problem, many ISPs use Carrier-Grade NAT.

With CGNAT, the ISP places multiple customers behind one or more shared public IPv4 addresses.

A simple example is:



The customer can normally access the Internet without any problem because the connection is started from inside the network.

However, incoming connections from the Internet become difficult because the public IPv4 address belongs to the ISP's CGNAT system rather than directly to the customer's router.

This creates problems for services such as:

- Remote access VPN servers
- Self-hosted websites
- Remote desktop services
- CCTV remote access
- Game servers
- Internal office services

For example, if a WireGuard VPN server is running behind CGNAT, a remote user cannot simply connect to the customer's private IPv4 address because private addresses are not globally routable.

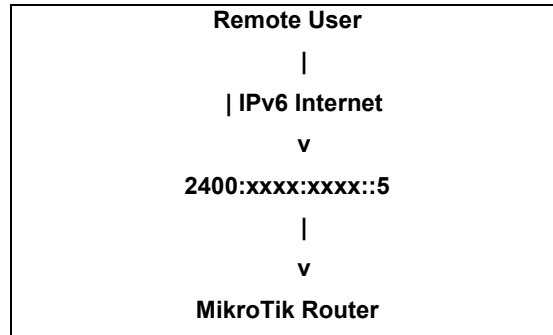
Some ISPs provide a public IPv4 address as an additional service, but this can involve extra cost.

2.2. How IPv6 Helps Remote Access

IPv6 was designed with a much larger address space than IPv4. Because of this, ISPs do not need to share addresses between customers in the same way that IPv4 CGNAT does.

An ISP can provide a globally routable IPv6 address directly to the customer's router.

For example:



The global IPv6 address can be used as the public endpoint of services such as WireGuard.

This means a remote WireGuard client can connect directly to the IPv6 address of the MikroTik router, provided that:

The ISP allows inbound IPv6 traffic.

- The MikroTik firewall allows the WireGuard UDP port.
- The remote network also has working IPv6 connectivity.
- The WireGuard endpoint is configured correctly.

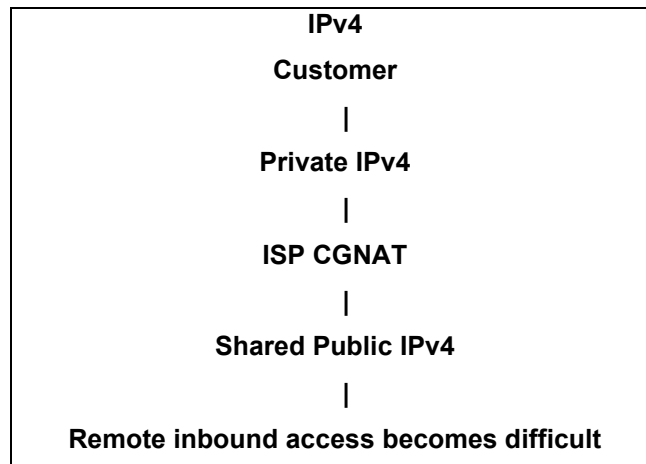
Unlike IPv4 CGNAT, there is no additional ISP NAT layer blocking the inbound connection in this design.

However, having a global IPv6 address does not automatically mean every service is publicly accessible. Firewall rules are still required to control which connections are allowed.

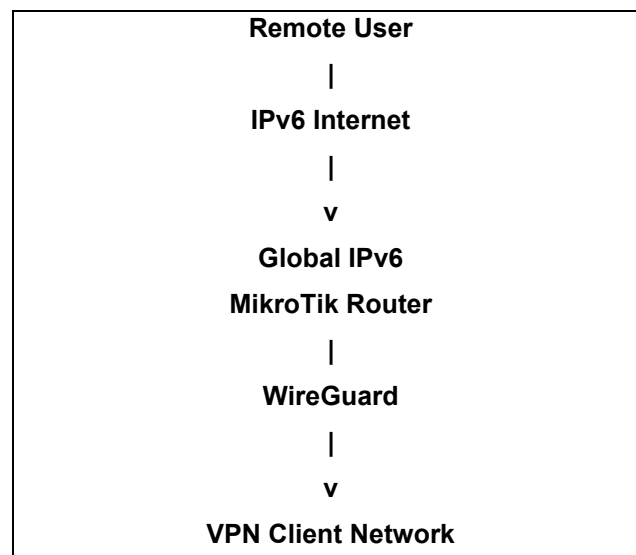
2.3. Why IPv6 Is Used in This Project

IPv6 is used in this project mainly to avoid depending on a public IPv4 address for the WireGuard VPN endpoint.

The main idea is:



Whereas with the IPv6 design:



The MikroTik router uses its global IPv6 address as the WireGuard server endpoint.

Inside the WireGuard tunnel, ULA IPv6 addresses are used for VPN clients. For example:

WireGuard Network
fd7a:115c:a1e0:92::/64

The MikroTik VPN gateway can use:

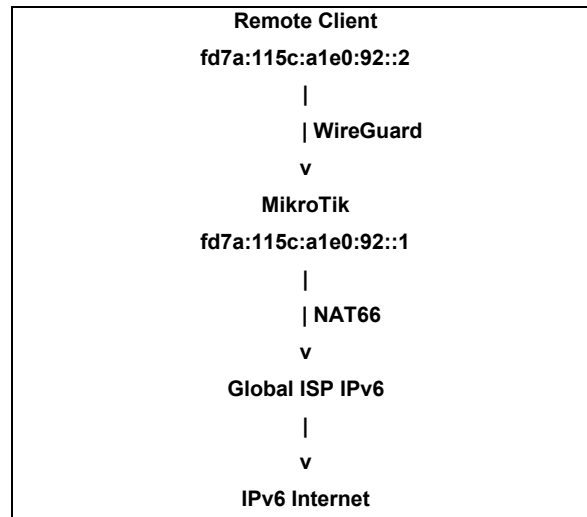
fd7a:115c:a1e0:92::1

and the first client can use:

fd7a:115c:a1e0:92::2

Because ULA addresses are not routable on the public Internet, NAT66 is used when VPN clients need IPv6 Internet access.

The final design is:



Therefore, IPv6 is important in this project because it provides the globally reachable address needed for the WireGuard endpoint while reducing the dependency on public IPv4 connectivity.

The project also demonstrates how this approach could provide a low-cost remote access solution for small and medium-sized businesses that already receive IPv6 connectivity from their ISP.

3. WireGuard and NAT66 Overview

This project uses WireGuard as the VPN technology and NAT66 to provide IPv6 Internet access to VPN clients using ULA addresses.

WireGuard creates the secure tunnel between the remote client and the MikroTik router. The client receives an internal IPv6 ULA address, while the MikroTik uses its global IPv6 address as the public VPN endpoint. NAT66 is then used to translate the ULA source address when the client needs to access the IPv6 Internet.

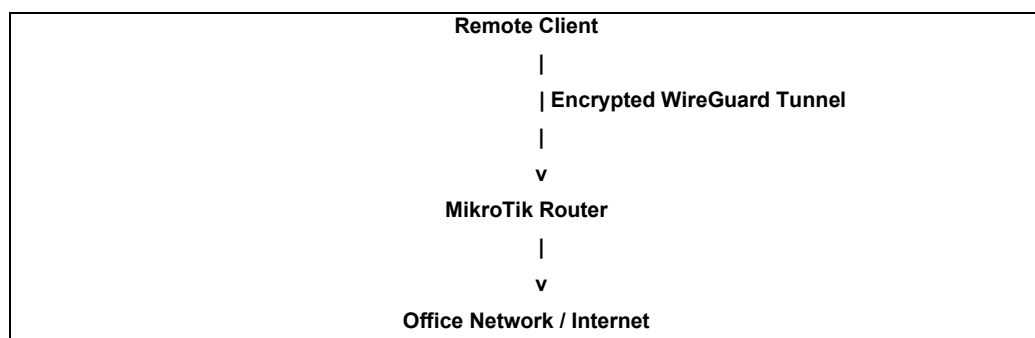
3.1. WireGuard VPN

WireGuard is a modern VPN protocol designed to provide secure and lightweight encrypted communication between devices.

It works by using public and private cryptographic keys. Each WireGuard device has its own key pair, and the public key is shared with the other peer.

In this project, the MikroTik router acts as the WireGuard server or VPN endpoint, while the remote device such as a phone or laptop acts as the client.

The basic connection is:



WireGuard normally uses UDP for communication. In this project, UDP port 51820 is used.

The main reasons for using WireGuard are:

- Simple configuration
- Strong encryption
- Good performance

- Low resource usage
- Support for IPv6
- Available directly in MikroTik RouterOS
- Suitable for remote-access VPN connections

The public IPv6 address of the MikroTik router is used as the WireGuard endpoint so that a remote client can connect to the router over the IPv6 Internet.

3.2. IPv6 ULA Addressing

ULA stands for Unique Local Address. It is an IPv6 address range designed for private or internal network communication.

ULA addresses normally use:

fc00::/7

In practice, locally generated ULA networks commonly start with:

fd

For this project, the following IPv6 network is used:

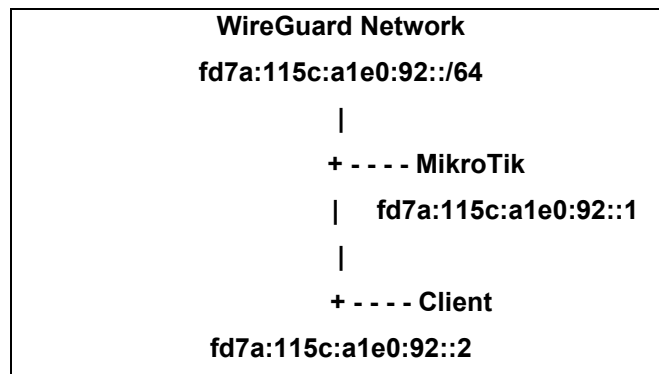
fd7a:115c:a1e0:92::/64

The addressing plan is:

MikroTik WireGuard Gateway
fd7a:115c:a1e0:92::1

Remote Client
fd7a:115c:a1e0:92::2

The basic layout is:



ULA addresses are useful because they allow the VPN network to have its own internal IPv6 addressing scheme.

However, ULA addresses are not globally routable on the public Internet. Because of this, another method is needed if the VPN client wants to access the IPv6 Internet.

This is where NAT66 is used.

3.3. NAT66

NAT66 stands for Network Address Translation between IPv6 addresses.

In this project, NAT66 is used to translate the source IPv6 address of the WireGuard client from its ULA address to the global IPv6 address available on the MikroTik WAN interface.

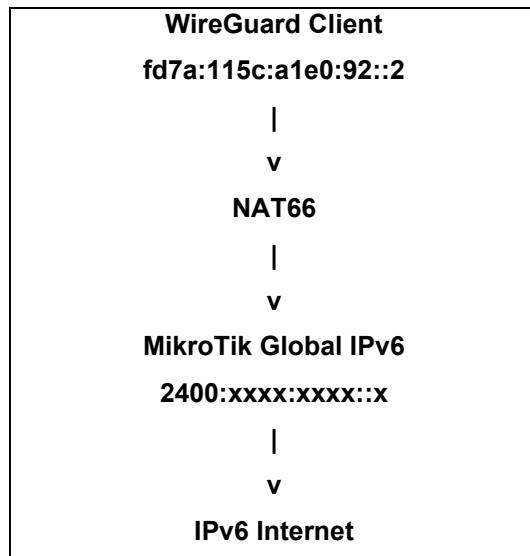
For example, the WireGuard client may send traffic using:

```
fd7a:115c:a1e0:92::/64
```

This address cannot be routed directly across the public IPv6 Internet.

The MikroTik translates it to the global IPv6 address available on its WAN interface.

The traffic flow is:



In this project, NAT66 is mainly required because the ISP provides a global IPv6 address to the MikroTik router but does not provide a usable delegated IPv6 prefix for assigning global addresses directly to the WireGuard clients.

If a usable global IPv6 prefix were delegated by the ISP, the clients could potentially use globally routable IPv6 addresses without NAT66.

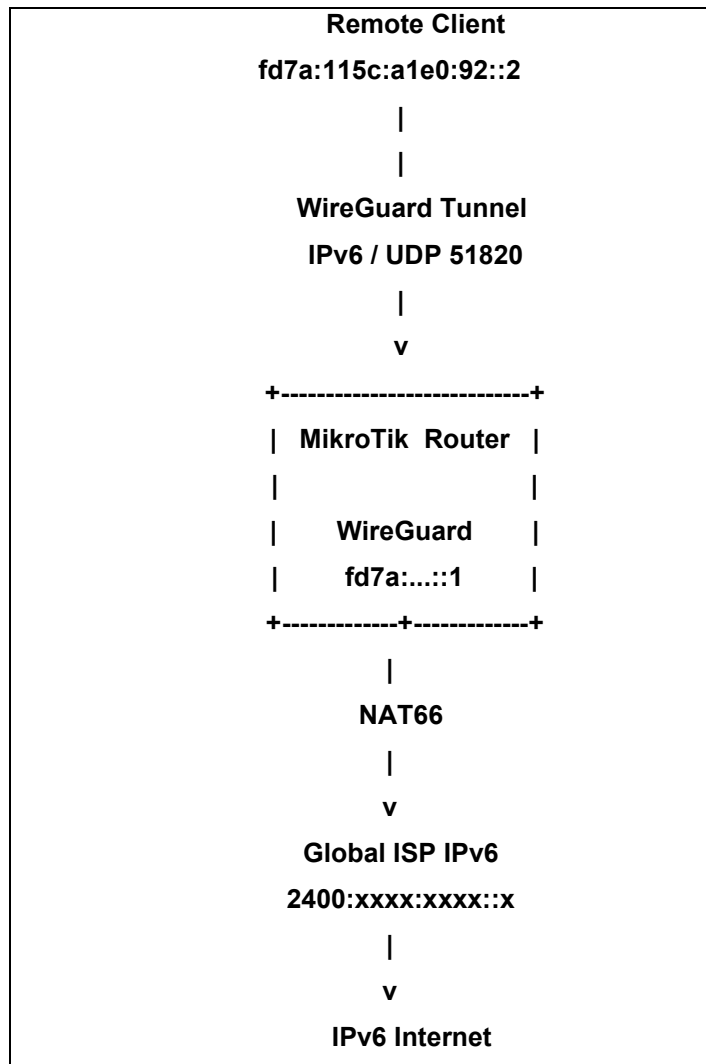
3.4. Proposed Solution

The proposed solution combines IPv6, WireGuard, ULA addressing, NAT66, and MikroTik RouterOS to create a low-cost remote-access VPN.

The MikroTik router receives a globally reachable IPv6 address from the ISP. This global IPv6 address is used as the public WireGuard endpoint.

The WireGuard clients use ULA IPv6 addresses inside the encrypted tunnel. When the client accesses the IPv6 Internet, NAT66 translates the ULA source address to the global IPv6 address of the MikroTik router.

The complete design is:



The main purpose of this solution is to avoid depending on a public IPv4 address for remote VPN access.

Instead, the project uses the globally routable IPv6 connectivity already available from the ISP to provide the WireGuard endpoint.

The proposed solution will first be implemented and tested in GNS3 using MikroTik CHR. The testing will verify IPv6 connectivity, WireGuard handshake, routing, firewall rules, NAT66 translation, and remote client communication before evaluating its suitability for small and medium-sized business environments.

4. Network Topology and Addressing Plan

This section explains the network design used for the IPv6-only WireGuard VPN project. The complete lab is created in GNS3 using MikroTik CHR.

The topology is kept simple so that the main focus remains on IPv6 connectivity, WireGuard, ULA addressing, NAT66, and remote access.

The main devices used are:

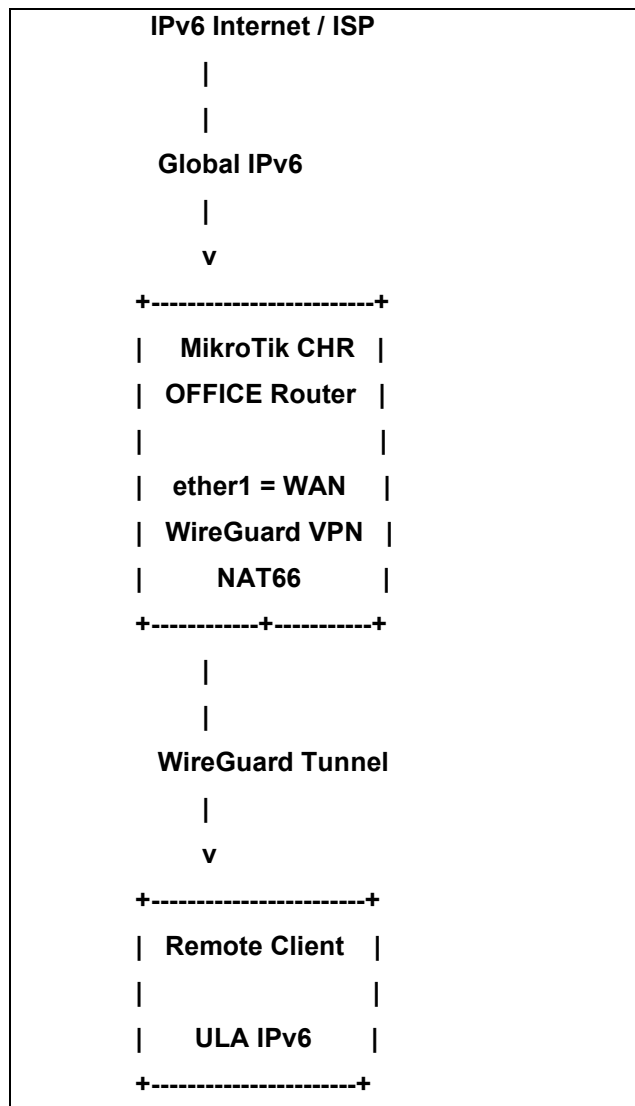
- ISP-side network
- MikroTik CHR router
- Remote WireGuard client
- IPv6 Internet / test destination

4.1. GNS3 Topology

The GNS3 topology is designed to represent a small office connected to an IPv6-capable ISP.

The MikroTik CHR router acts as the office gateway and WireGuard VPN endpoint. The remote client connects to the MikroTik router over IPv6.

The basic topology is:



In the lab, the MikroTik router performs the following functions:

- Receives or uses a global IPv6 address on the WAN side
- Acts as the WireGuard VPN endpoint
- Provides ULA IPv6 addressing for the VPN network
- Performs IPv6 routing
- Applies IPv6 firewall rules
- Performs NAT66 for VPN client Internet access

The remote client represents a user connecting from outside the office network.

4.2. IPv6 Addressing Plan

Two types of IPv6 addresses are used in this project:

- Global IPv6 address for the WAN and WireGuard endpoint
- ULA IPv6 addresses for the WireGuard tunnel

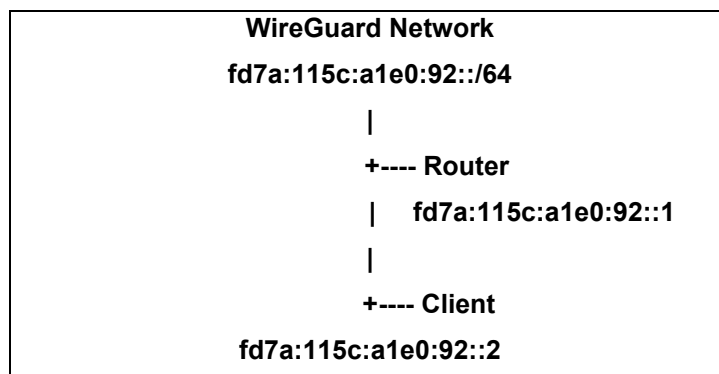
For the WireGuard network, the following ULA prefix is used:

fd7a:115c:a1e0:92::/64

The addressing plan is:

Device	Interface	IPv6 Address	Purpose
MikroTik CHR	WAN / ether1	Global IPv6	Public WireGuard endpoint
MikroTik CHR	WG-VPN-IPv6	fd7a:115c:a1e0:92::1/64	VPN gateway
Remote Client	WireGuard	fd7a:115c:a1e0:92::2/128	VPN client

The address structure can be shown as:



The global IPv6 address is used only as the externally reachable VPN endpoint.

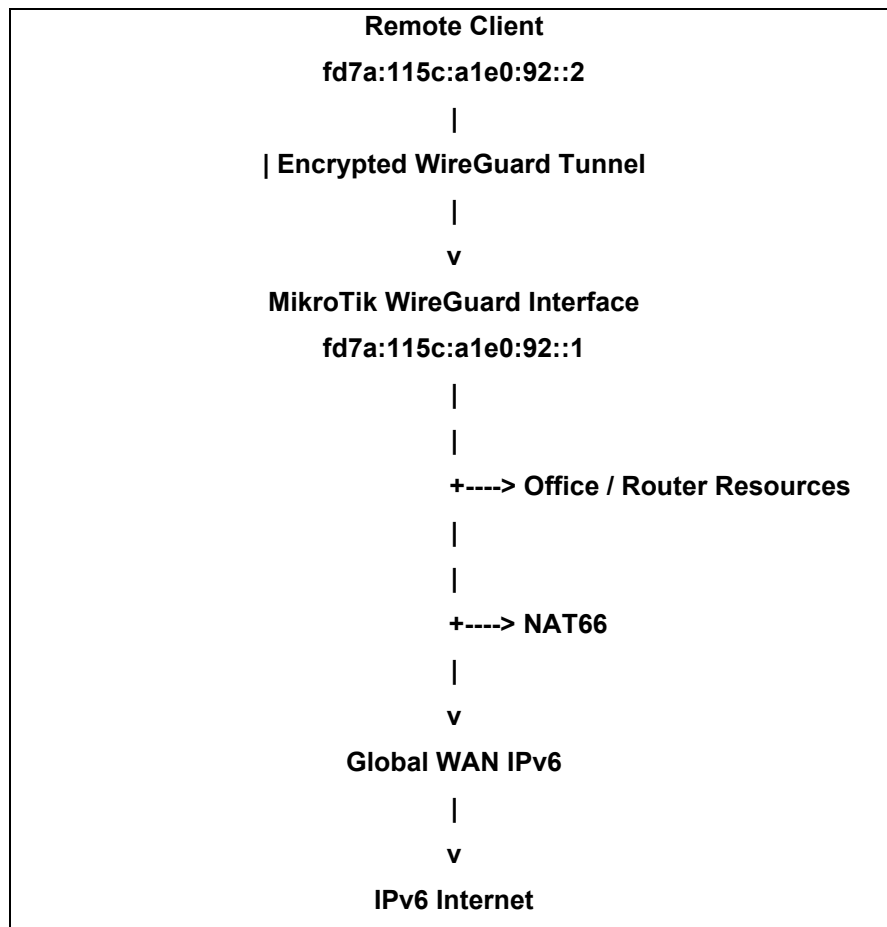
The ULA addresses are used for communication inside the WireGuard tunnel.

4.3. Traffic Flow

The remote client first connects to the global IPv6 address of the MikroTik router using WireGuard over UDP port 51820.

After the WireGuard tunnel is established, the client uses its ULA IPv6 address inside the VPN.

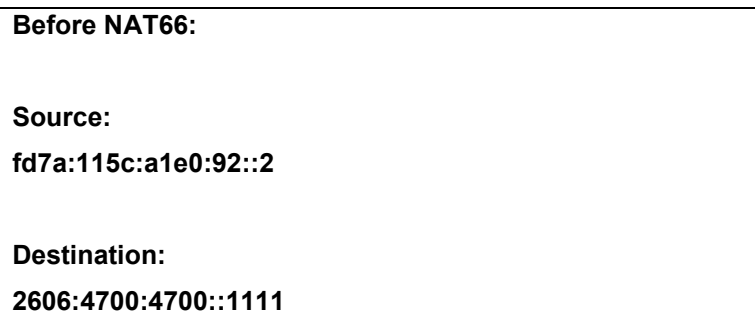
The traffic flow is:



When the client accesses internal resources, the traffic stays inside the VPN and office network.

When the client accesses the IPv6 Internet, the ULA source address is translated using NAT66.

For example:

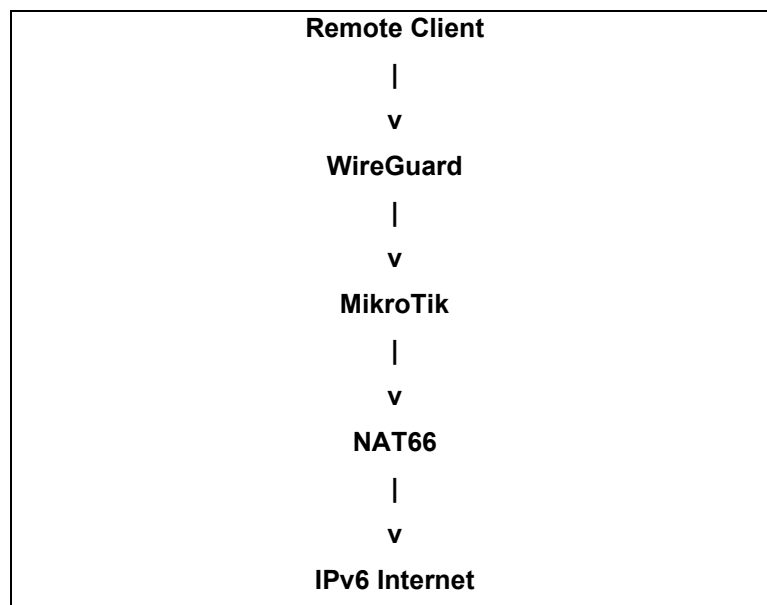


After NAT66:

Source: MikroTik Global IPv6
Destination: 2606:4700:4700::1111

The return traffic is then translated back and sent through the WireGuard tunnel to the remote client.

So the complete path is:



This design allows the remote client to securely connect to the MikroTik router through IPv6 and use the office connection without depending on a public IPv4 address.

5. IPv6 WAN Configuration

Before configuring WireGuard, the MikroTik router must first have working IPv6 connectivity. In this project, ether1 is used as the WAN interface and the router receives its global IPv6 address using DHCPv6.

This section covers the IPv6 WAN configuration, verification of the global IPv6 address, checking the default IPv6 route, and finally testing IPv6 Internet connectivity.

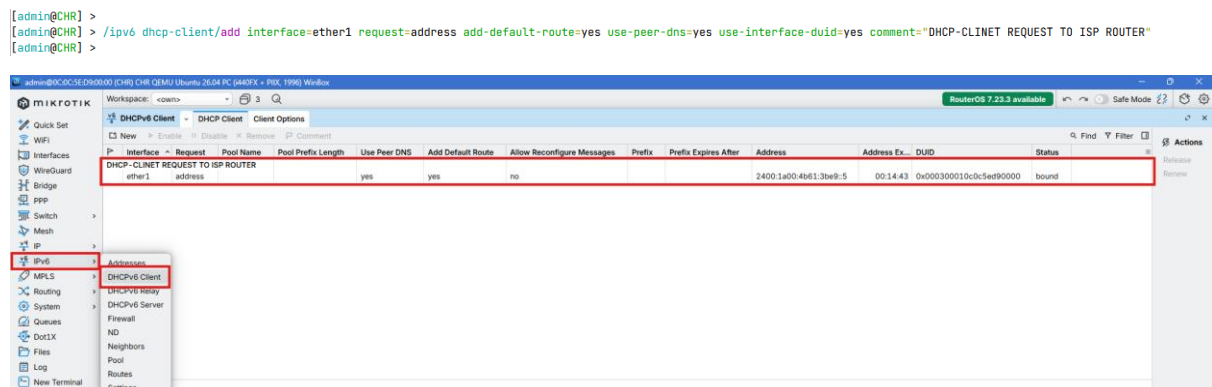
5.1. Configure IPv6 WAN

A DHCPv6 client is configured on ether1 so that the MikroTik router can request an IPv6 address from the upstream ISP/network.

The option add-default-route=yes automatically adds the IPv6 default route after the DHCPv6 client successfully connects.

CMD

```
/ipv6 dhcp-client/add interface=ether1 request=address add-default-route=yes use-peer-dns=yes use-interface-duid=yes comment="DHCP-CLINET REQUEST TO ISP ROUTER"
```



CMD

```
/ipv6 dhcp-client print detail
```

The configuration can be checked using:

```
[admin@CHR] > /ipv6 dhcp-client/print detail
Flags: D - DYNAMIC; X - DISABLED; I - INVALID
0      ;; DHCP-CLINET REQUEST TO ISP ROUTER
interface=ether1 status=bound duid="0x000300010c0c5e990000" dhcp-server-v6-fe80::c254:4dff:fe98:b608 request=address accept-prefix-without-address=yes add-default-route=yes default-route-distance=1
default-route-tables=default check-gateway=no use-peer-dns=yes use-interface-duid=yes validate-server-duid=yes allow-reconfigure=no dhcp-options="" pool-name="" prefix-hint::/0 prefix-address-lists=""
dhcp-options="" address=2400:1a00:4b61:3be9::5, 19a42s
[admin@CHR] >
```

A successful connection should show: **status=bound** and an IPv6 address received from the upstream network.

For example:

```
[admin@CHR] > /ip6 dhcp-client/print detail
[admin@CHR] > /ip6 dhcp-client/print detail
Flags: D - DYNAMIC; X - DISABLED; I - INVALID
0   DHCP-CLIENT REQUEST TO ISP ROUTER
    interface-ether1 status=bound duid="0x000300010c0c5e990000" dhcp-server-v6-fe80::c254:4dff:fe98:b608 request-address accept-prefix-without-address=yes add-default-route=yes default-route-distance=1
    interface-ether1 status=bound duid="0x000300010c0c5e990000" use-peer-dns=yes use-interface-duid=yes validate-server-duid=yes allow-reconfigure=no dhcp-options="" pool-name="" prefix-hint::/8 prefix-address-lists=""
    default-route-table=default check-nat=yes use-peer-dns=yes use-interface-duid=yes validate-server-duid=yes allow-reconfigure=no dhcp-options="" pool-name="" prefix-hint::/8 prefix-address-lists=""
    dhcp-options="" address=2400:1a00:4b61:3be9::5/128 19m42s
[admin@CHR] >
```

This confirms that the DHCPv6 client is working correctly.

5.2. Verify Global IPv6 Address

After the DHCPv6 client is successfully connected, the next step is to verify that the MikroTik router has received a Global Unicast IPv6 Address.

The following command is used:

```
/ip6 address print detail
```

The router normally has more than one IPv6 address.

For example:

```
[admin@CHR] > /ip6 address/print detail
[admin@CHR] > /ip6 address/print detail
Flags: X - DISABLED; I - INVALID; D - DYNAMIC; G - GLOBAL; L - LINK-LOCAL; S - SLAVE; d - DEPRECATED
0   D   address::1/128 from-pool="" interface=lo actual-interface=lo vrf=main eui-64=no advertise=no no-dad=no auto-link-local=yes
1   DL  address=fe80::e0c:5eff:fed9:0/64 from-pool="" interface=ether1 actual-interface=ether1 vrf=main eui-64=no advertise=no no-dad=no auto-link-local=yes
2   DG  address=2400:1a00:4b61:3be9::5/128 from-pool="" interface=ether1 actual-interface=ether1 vrf=main eui-64=no advertise=no no-dad=no auto-link-local=yes
[admin@CHR] >
```

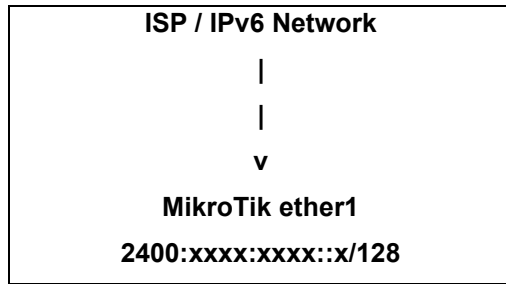
is a link-local IPv6 address, while an address similar to:

```
[admin@CHR] > /ip6 address/print detail
[admin@CHR] > /ip6 address/print detail
Flags: X - DISABLED; I - INVALID; D - DYNAMIC; G - GLOBAL; L - LINK-LOCAL; S - SLAVE; d - DEPRECATED
0   D   address::1/128 from-pool="" interface=lo actual-interface=lo vrf=main eui-64=no advertise=no no-dad=no auto-link-local=yes
1   DL  address=fe80::e0c:5eff:fed9:0/64 from-pool="" interface=ether1 actual-interface=ether1 vrf=main eui-64=no advertise=no no-dad=no auto-link-local=yes
2   DG  address=2400:1a00:4b61:3be9::5/128 from-pool="" interface=ether1 actual-interface=ether1 vrf=main eui-64=no advertise=no no-dad=no auto-link-local=yes
[admin@CHR] >
```

is the global IPv6 address received from the upstream network.

In this project, the global IPv6 address is important because it will later be used as the public WireGuard endpoint.

The WAN side can therefore be represented as:



The actual global IPv6 address may change depending on the address provided by the ISP or simulated upstream network.

5.3. Verify IPv6 Default Route

Having a global IPv6 address alone is not enough to access the IPv6 Internet. The router also requires a default IPv6 route.

The IPv6 default route is:

```
::/0
```

This works similarly to:

```
0.0.0.0/0
```

in IPv4.

The routing table can be checked using:

CMD

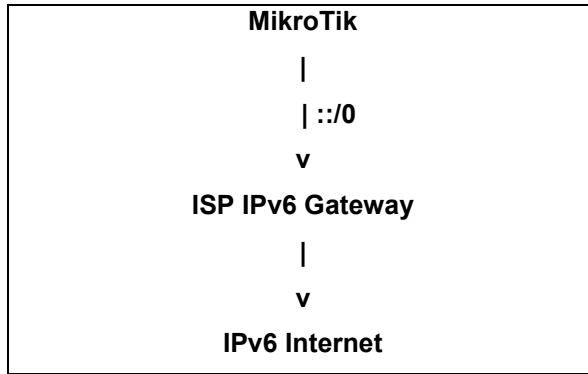
```
/ipv6 route print detail
```

A successful configuration should show an active route similar to:

```

[admin@CHR] > /ipv6 route/print detail
Flags: D - DYNAMIC; X - DISABLED; I - INACTIVE; A - ACTIVE; C - CONNECT; S - STATIC; R - RTP; B - BGP; O - OSPF; T - TS-TS; d - DHCP; v - VPN; m - MODEM; p - SLAAC; y - BGP-MPLS-VPN; H - HW-OFFLOADED; + - ECMP
Dad  dst-address::/0 routing-table=main gateway=fe80::c254:4dff:fe98:b688ether1 immediate-gw=fe80::c254:4dff:fe98:b688ether1 distance=1 scope=30 target-scope=10 vrf-interface=ether1
Dac  dst-address::1/128 routing-table=main gateway=lo immediate-gw=lo distance=0 scope=10 target-scope=5
Dac  dst-address=2400:1a00:4b61:3be9::5/128 routing-table=main gateway=ether1 immediate-gw=ether1 distance=0 scope=10 target-scope=5
Dac  dst-address=fe80::/64 routing-table=main gateway=ether1 immediate-gw=ether1 distance=0 scope=10 target-scope=5
[admin@CHR] >
  
```

The traffic flow is:



If the `::/0` route is active, the MikroTik knows where to forward traffic for external IPv6 destinations.

5.4. Test IPv6 Connectivity

After confirming the IPv6 address and default route, the final step is to test IPv6 connectivity.

Cloudflare's IPv6 DNS address can be used for testing:

CMD

```
/ping 2606:4700:4700::1111 count=5
```

A successful result should look similar to:

```
[admin@CHR] > /ping 2606:4700:4700::1111 count=5
SEQ HOST                                SIZE TTL TIME                    STATUS
0 2606:4700:4700::1111                  56  58 6ms827us  echo reply
1 2606:4700:4700::1111                  56  58 2ms349us  echo reply
2 2606:4700:4700::1111                  56  58 5ms215us  echo reply
3 2606:4700:4700::1111                  56  58 2ms583us  echo reply
4 2606:4700:4700::1111                  56  58 2ms466us  echo reply
sent=5 received=5 packet-loss=0% min-rtt=2ms349us avg-rtt=3ms888us max-rtt=6ms827us
[admin@CHR] >
```

Google's IPv6 DNS address can also be tested:

CMD

```
/ping 2001:4860:4860::8888 count=5
```

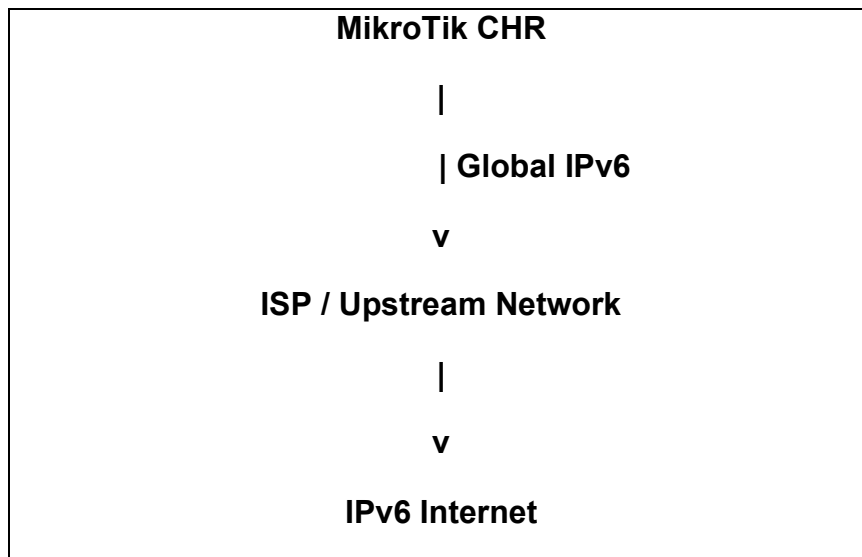
A successful result should look like:

```
[admin@CHR] >
[admin@CHR] > /ping 2001:4860:4860::8888 count=5
  SEQ HOST                                SIZE TTL TIME          STATUS
  --- --
0 2001:4860:4860::8888                    56 115 19ms241us  echo reply
1 2001:4860:4860::8888                    56 115 19ms285us  echo reply
2 2001:4860:4860::8888                    56 115 17ms622us  echo reply
3 2001:4860:4860::8888                    56 115 18ms249us  echo reply
4 2001:4860:4860::8888                    56 115 17ms792us  echo reply
sent=5 received=5 packet-loss=0% min-rtt=17ms622us avg-rtt=18ms437us max-rtt=19ms285us

[admin@CHR] > █
```

At this stage, the IPv6 WAN connection is working successfully.

The network currently looks like:



With the IPv6 WAN successfully configured and tested, the next step is to configure the WireGuard IPv6 VPN interface and ULA addressing.

6. WireGuard IPv6 Configuration

After confirming that the MikroTik router has working IPv6 connectivity, the next step is to configure the WireGuard VPN.

In this project, the global IPv6 address on the WAN interface is used as the public WireGuard endpoint. A separate ULA IPv6 network is used inside the VPN tunnel for communication between the MikroTik router and remote clients.

The WireGuard network used in this project is:

```
fd7a:115c:a1e0:92::/64
```

The addressing is:

```
MikroTik WireGuard Gateway: fd7a:115c:a1e0:92::1
```

```
Remote Client: fd7a:115c:a1e0:92::2
```

6.1. Create WireGuard Interface

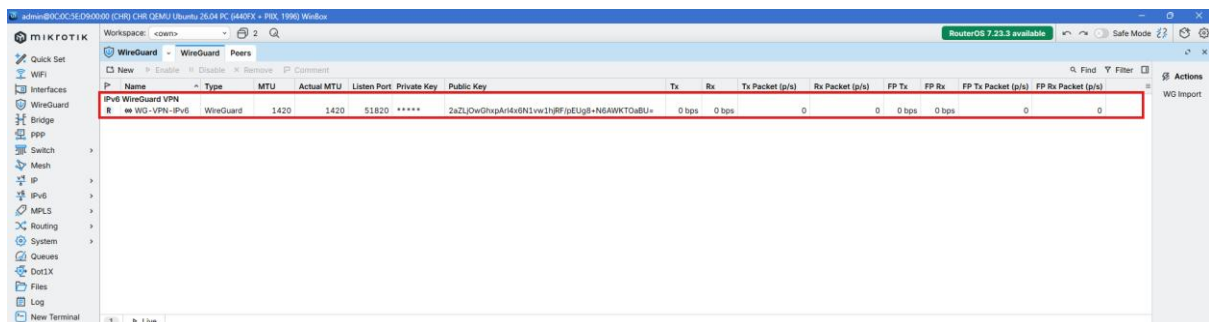
First, a WireGuard interface is created on the MikroTik router.

In this project, UDP port 51820 is used for the WireGuard connection.

CMD

```
interface/wireguard/add name=WG-VPN-IPv6 listen-port=51820 mtu=1420 comment="IPv6 WireGuard VPN"
```

```
[admin@CHR] >
[admin@CHR] > interface/wireguard/add name=WG-VPN-IPv6 listen-port=51820 mtu=1420 comment="IPv6 WireGuard VPN"
[admin@CHR] >
```

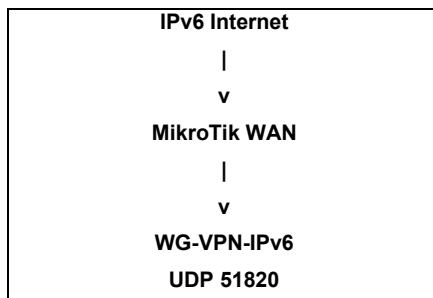


```
[admin@CHR] >
[admin@CHR] > /interface/wireguard/print detail
Flags: X - DISABLED; R - RUNNING
0 R ::: IPv6 WireGuard VPN
  name="WG-VPN-IPv6" mtu=1420 listen-port=51820 public-key="2aZLj0wGhxpArL4x6N1vw1hjRF/pEUg8+N6AWKT0aBU="
[admin@CHR] >
```

When the WireGuard interface is created, MikroTik automatically generates the router's WireGuard private and public keys.

The private key should always be kept secret.

The WireGuard interface now acts as the VPN interface:



6.2. Assign ULA IPv6 Address

After creating the WireGuard interface, an IPv6 address must be assigned to it.

For this project, the following ULA prefix is used:

```
fd7a:115c:a1e0:92::/64
```

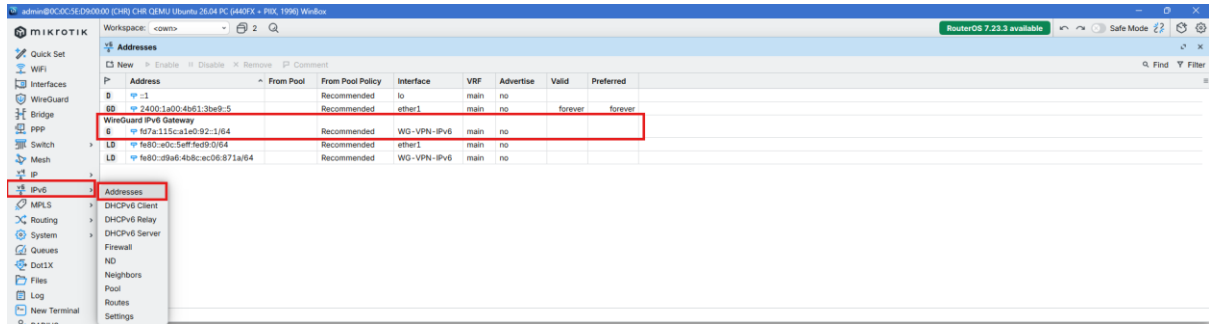
The MikroTik WireGuard interface uses:

```
fd7a:115c:a1e0:92::1/64
```

CMD

```
/ipv6 address/ add address=fd7a:115c:a1e0:92::1/64 interface=WG-VPN-IPv6 advertise=no comment="WireGuard IPv6 Gateway"
```

```
[admin@CHR] >
[admin@CHR] > /ipv6 address/ add address=fd7a:115c:a1e0:92::1/64 interface=WG-VPN-IPv6 advertise=no comment="WireGuard IPv6 Gateway"
[admin@CHR] >
```



The address can be verified using:

CMD

```
/ip address print detail where interface=WG-VPN-IPv6
```

The expected address is:

```
[admin@CHR] > /ip address print detail where interface=WG-VPN-IPv6
Flags: X - DISABLED, I - INVALID, D - DYNAMIC, G - GLOBAL, L - LINK-LOCAL, S - SLAVE, d - DEPRECATED
3 DL address=fe80::d9a6:4b8c:ec86:871a/64 from-pool="" interface=WG-VPN-IPv6 actual-interface=WG-VPN-IPv6 vrf=main eui-64=no advertise=no no-dad=no auto-link-local=yes
4 G ;; WireGuard IPv6 Gateway
address=fd7a:115c:a1e0:92::1/64 from-pool="" interface=WG-VPN-IPv6 actual-interface=WG-VPN-IPv6 vrf=main eui-64=no advertise=no no-dad=no auto-link-local=yes
[admin@CHR] >
```

The WireGuard VPN network is now:

```
fd7a:115c:a1e0:92::/64
|
+---- MikroTik
      fd7a:115c:a1e0:92::1
```

Later, each WireGuard client will receive its own IPv6 address from this network.

6.3. Create WireGuard Peer

A WireGuard peer represents the remote client that is allowed to connect to the VPN.

For the first client, the IPv6 address used is:

```
fd7a:115c:a1e0:92::2/128
```

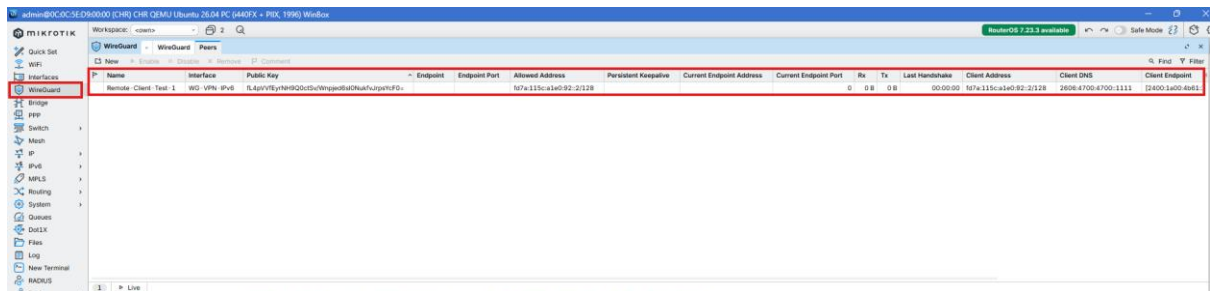
In this project, MikroTik generates the client's WireGuard key automatically using:
private-key=auto

Replace **<GLOBAL-IPv6-ADDRESS>** with the global IPv6 address available on the MikroTik WAN interface. On my case I have “2400:1a00:4b61:3be9::5/128”.

CMD

```
/interface/wireguard/peers/add interface=WG-VPN-IPv6 name=Remote-Client-Test-1 private-key=auto allowed-address=fd7a:115c:a1e0:92::2/128 client-address=fd7a:115c:a1e0:92::2/128 client-allowed-address=::/0 client-dns=2606:4700:4700::1111 client-endpoint="[2400:1a00:4b61:3be9::5]" client-keepalive=25 responder=yes
```

```
[admin@CHR] > /interface/wireguard/peers/add interface=WG-VPN-IPv6 name=Remote-Client-Test-1 private-key=auto allowed-address=fd7a:115c:a1e0:92::2/128 client-address=fd7a:115c:a1e0:92::2/128 client-allowed-address=::/0 client-dns=2606:4700:4700::1111 client-endpoint="[2400:1a00:4b61:3be9::5]" client-keepalive=25 responder=yes
```



the client endpoint would be:

```
"[2400:1a00:4b61:3be9::5]"
```

The brackets are important when an IPv6 address is used as a WireGuard endpoint.

The peer can be verified using:

```
/interface wireguard peers print detail
```

```
[admin@CHR] > /interface/wireguard/peers/print detail
Flags: X - DISABLED, D - DYNAMIC
0 interface=WG-VPN-IPv6 name="Remote-Client-Test-1" public-key="fL4pVVfEYrNH9Q0ctSv/Wnpjed6sI0NukfVJrpsYcF0=" endpoint-address="" endpoint-port=0 current-endpoint-address="" current-endpoint-port=0
allowed-address=fd7a:115c:a1e0:92::2/128 client-address=fd7a:115c:a1e0:92::2/128 client-dns=2606:4700:4700::1111 client-endpoint="[2400:1a00:4b61:3be9::5]" client-keepalive=25s
client-allowed-address=::/0 responder=yes rx=0 tx=0
```

The important values should include:

```
allowed-address=fd7a:115c:a1e0:92::2/128
client-address=fd7a:115c:a1e0:92::2/128
client-allowed-address=::/0
client-keepalive=25s
```

The server-side **allowed-address** tells MikroTik that:

```
fd7a:115c:a1e0:92::2
```

belongs to this specific WireGuard peer.

6.4. Configure Client

The remote WireGuard client needs its own IPv6 address, private key, MikroTik public key, and the global IPv6 address of the MikroTik router.

The client configuration follows this structure:

```
[Interface]

PrivateKey = <CLIENT-PRIVATE-KEY>

Address = fd7a:115c:a1e0:92::2/128

DNS = 2606:4700:4700::1111


[Peer]

PublicKey = <MIKROTIK-PUBLIC-KEY>

AllowedIPs = ::/0

Endpoint = [<GLOBAL-IPv6-ADDRESS>]:51820

PersistentKeepalive = 25
```

Only case

```
[Interface]

PrivateKey = <your-private-key>

Address = fd7a:115c:a1e0:92::2/128

DNS = 2606:4700:4700::1111

ListenPort = 51820


[Peer]

PublicKey = 2aZLjOwGhxpArI4x6N1vw1hjRF/pEUg8+N6AWKTOaBU=

AllowedIPs = ::/0

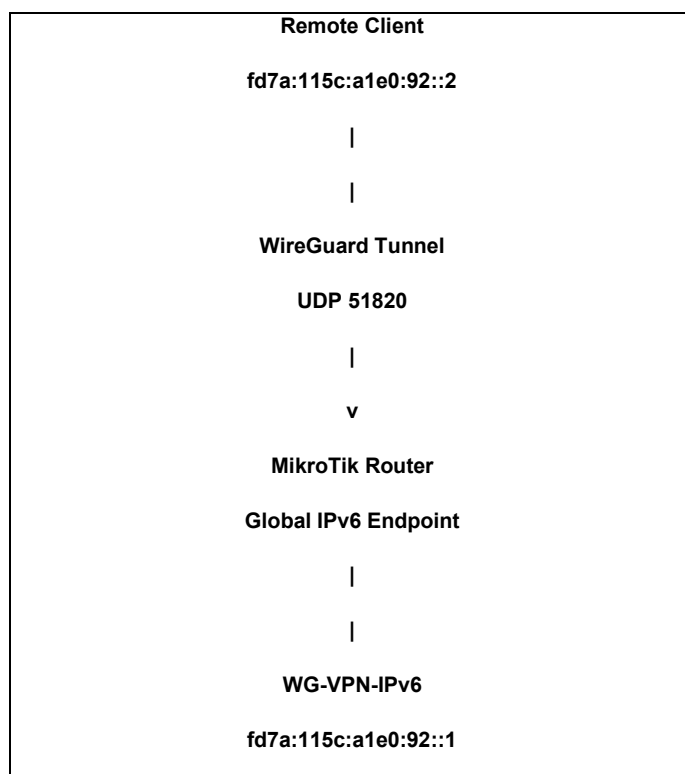
Endpoint = [2400:1a00:4b61:3be9::5]:51820

PersistentKeepalive = 25
```


The following line: **AllowedIPs = ::/0** means that all IPv6 traffic from the client is sent through the WireGuard VPN.

The endpoint: **[GLOBAL-IPv6]:51820** tells the client where the MikroTik WireGuard server is available on the Internet.

The final WireGuard structure is:



At this stage, the WireGuard interface, IPv6 tunnel addressing, and remote client peer are configured.

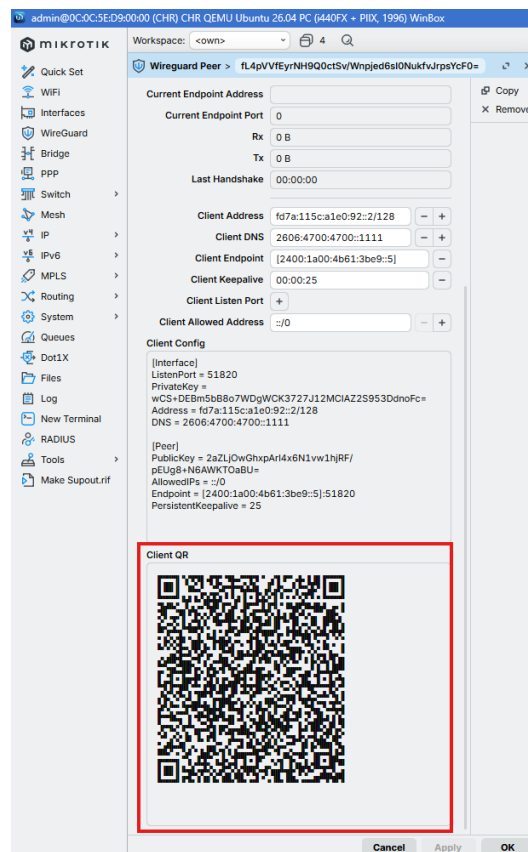
The next step is to configure the IPv6 firewall rules so that the WireGuard connection and VPN traffic are allowed securely.

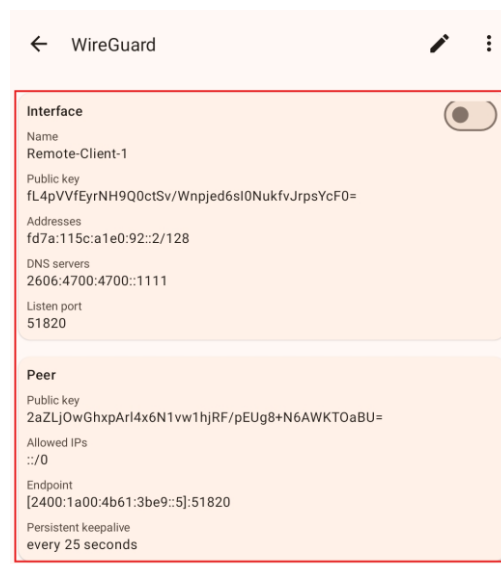
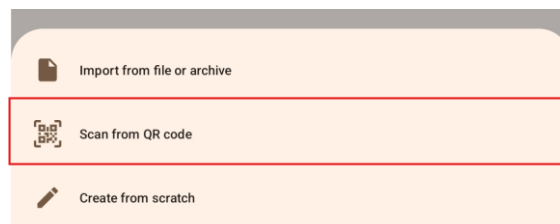
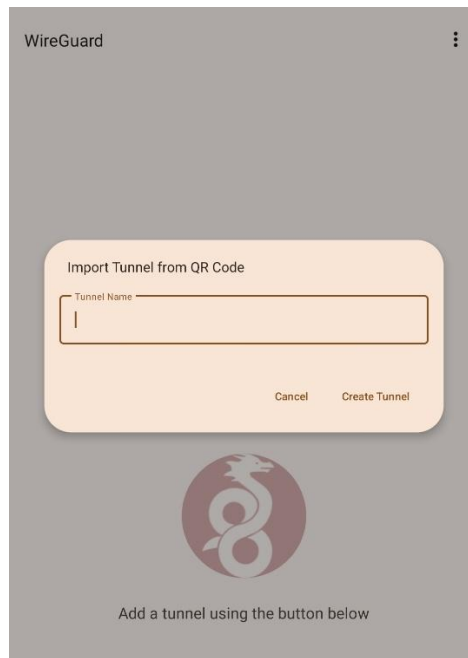
6.5. Generate Client Configuration / QR Code

After creating the WireGuard peer, MikroTik RouterOS can generate the complete client configuration automatically. This makes it easier to configure a mobile phone because the configuration can be imported directly by scanning a QR code.

The generated client configuration contains:

- Client private key
- Client IPv6 address
- DNS server
- MikroTik WireGuard public key
- Allowed IPv6 routes
- Global IPv6 endpoint
- WireGuard UDP port
- Persistent keepalive value





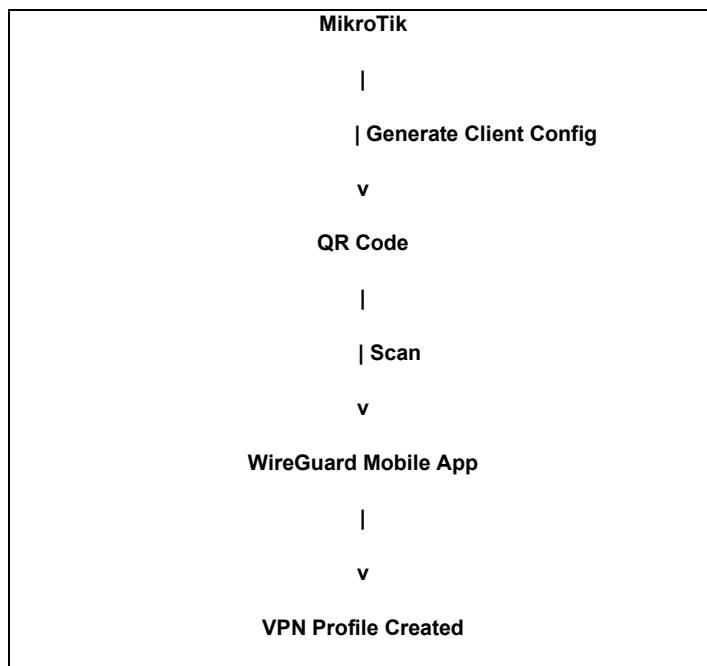
The important endpoint format for IPv6 is: **[IPv6-ADDRESS]:PORT**

For example:

[2400:1a00:4b61:3be9::5]:51820

The square brackets are required because IPv6 addresses already contain colons.

The QR code can then be scanned using the WireGuard mobile application:

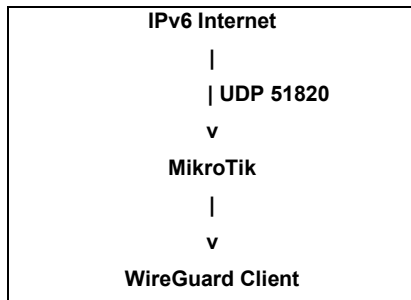


The QR code and private key should not be included in public documentation or screenshots because they contain sensitive client credentials.

7. IPv6 Firewall Configuration

After configuring WireGuard, IPv6 firewall rules are required to allow the VPN connection and control the traffic passing through the MikroTik router.

The firewall configuration in this section allows:



It also allows the authenticated WireGuard client to communicate with the MikroTik router and forward IPv6 traffic through the VPN.

7.1. Allow WireGuard Traffic

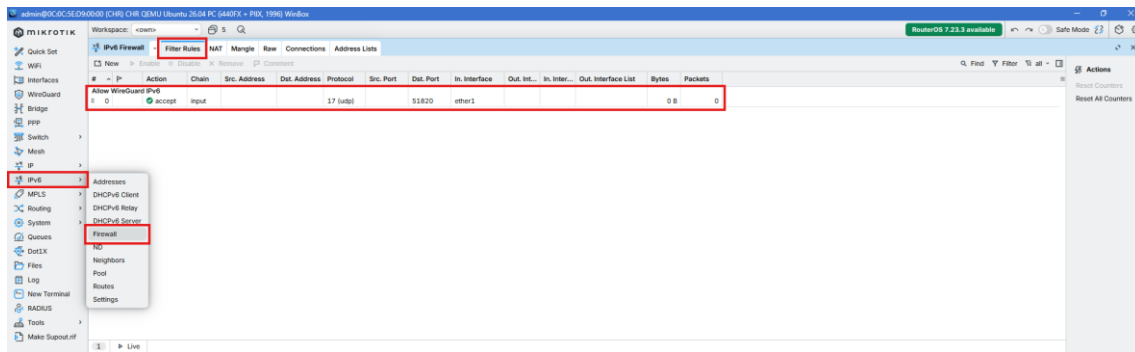
WireGuard uses UDP port 51820 in this project.

The MikroTik router must allow incoming IPv6 traffic to this port on the WAN interface. Without this rule, the remote client may not be able to establish the WireGuard connection when a restrictive firewall is used.

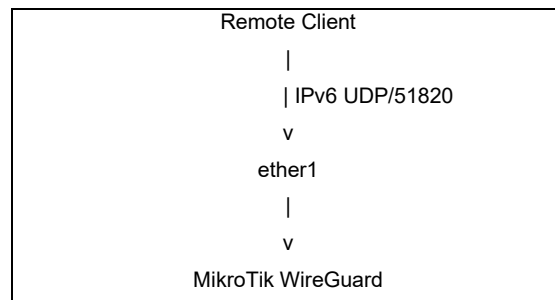
CMD

```
/ipv6 firewall/filter/add chain=input action=accept protocol=udp dst-port=51820 in-interface=ether1
comment="Allow WireGuard IPv6"
```

```
[admin@CHR] >
[admin@CHR] > /ipv6 firewall/filter/add chain=input action=accept protocol=udp dst-port=51820 in-interface=ether1 comment="Allow WireGuard IPv6"
[admin@CHR] >
```



The traffic flow is:



The firewall rule can be verified using:

CMD

```
/ipv6 firewall filter print detail
```

```

[admin@CHR] >
[admin@CHR] > /ipv6 firewall/filter/print detail
Flags: X - DISABLED, I - INVALID, D - DYNAMIC
0      ;; Allow WireGuard IPv6
chain=input action=accept protocol=udp in-interface=ether1 dst-port=51820
[admin@CHR] >
  
```

7.2. Allow VPN Client Traffic

After the WireGuard tunnel is established, the VPN client may need to communicate with the MikroTik router itself.

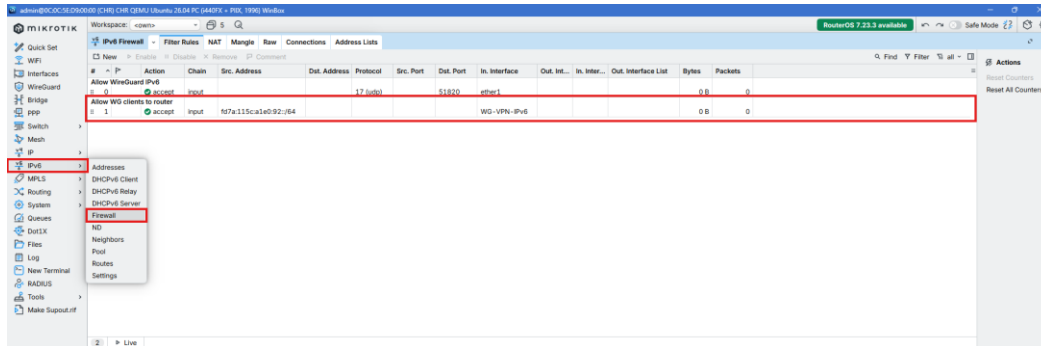
The WireGuard client network used in this project is: **fd7a:115c:a1e0:92::/64**

The following rule allows traffic coming from the WireGuard VPN network to the MikroTik router.

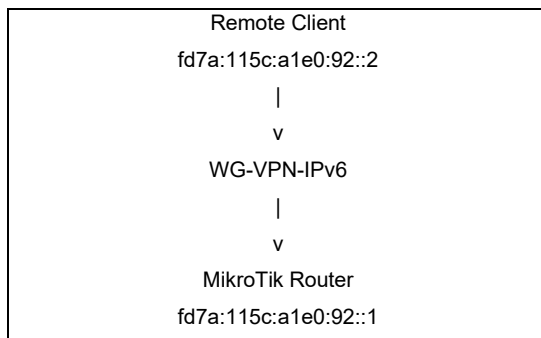
CMD

```
/ipv6 firewall/filter/add chain=input action=accept src-address=fd7a:115c:a1e0:92::/64 in-interface=WG-VPN-IPv6
comment="Allow WG clients to router"
```

```
[admin@CHR] >
[admin@CHR] > /ipv6 firewall/filter/add chain=input action=accept src-address=fd7a:115c:a1e0:92::/64 in-interface=WG-VPN-IPv6 comment="Allow WG clients to router"
[admin@CHR] >
```



This allows communication such as:



For example, the client should be able to reach the WireGuard gateway:
fd7a:115c:a1e0:92::1

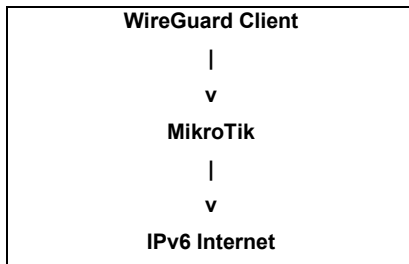
This rule is mainly used during the lab to allow VPN client-to-router communication. In a production environment, access should be restricted only to the management services that are actually required.

7.3. Allow IPv6 Forwarding

The previous rule allows the WireGuard client to communicate with the router itself. However, another rule is required when the client's traffic needs to pass through the MikroTik router.

This is handled by the forward chain.

For example:

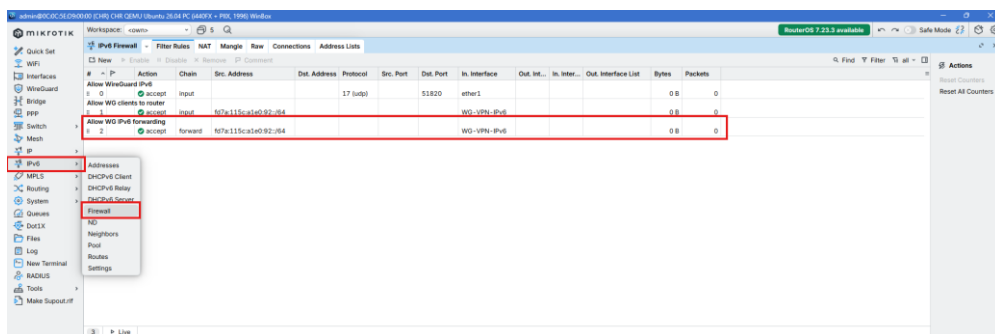


The following rule allows IPv6 traffic from the WireGuard network to be forwarded by the router.

CMD

```
/ipv6/firewall/filter/add chain=forward action=accept src-address=fd7a:115c:a1e0:92::/64 in-interface=WG-VPN-IPv6
comment="Allow WG IPv6 forwarding"
```

```
[admin@CHR] > /ipv6/firewall/filter/add chain=forward action=accept src-address=fd7a:115c:a1e0:92::/64 in-interface=WG-VPN-IPv6 comment="Allow WG IPv6 forwarding"
[admin@CHR] >
```



The rule can be verified using:

CMD

```
/ipv6 firewall filter print detail
```

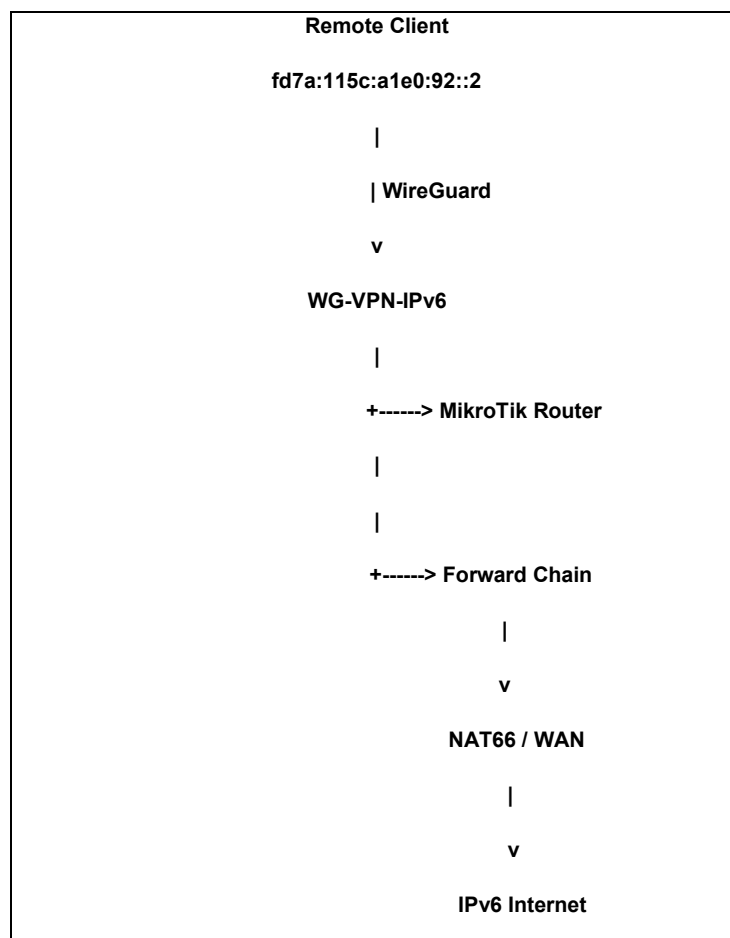
```

[admin@CHR] >
[admin@CHR] > /ipv6 firewall filter print detail
Flags: X - DISABLED, I - INVALID; D - DYNAMIC
0 ... Allow WireGuard IPv6
  chain=input action=accept protocol=udp in-interface=ether1 dst-port=51820
1 ... Allow WG clients to router
  chain=input action=accept src-address=fd7a:115c:a1e0:92::/64 in-interface=WG-VPN-IPv6
2 ... Allow WG IPv6 forwarding
  chain=forward action=accept src-address=fd7a:115c:a1e0:92::/64 in-interface=WG-VPN-IPv6
[admin@CHR] >
  
```


The final firewall rules should appear similar to:

1. Allow UDP 51820 from WAN
2. Allow WireGuard clients to MikroTik
3. Allow WireGuard clients to forward IPv6 traffic

The traffic flow is:



At this stage, the WireGuard connection and IPv6 forwarding are permitted by the firewall.

The next step is to configure NAT66, which will translate the WireGuard client's ULA IPv6 address when accessing the public IPv6 Internet.

8. NAT66 Configuration

After the WireGuard tunnel is configured, the VPN client can communicate with the MikroTik router using its ULA IPv6 address. However, ULA addresses are not globally routable on the public Internet.

For this reason, NAT66 is used so that IPv6 traffic from the WireGuard client can be translated to the global IPv6 address available on the MikroTik WAN interface.

8.1. Why NAT66 Is Required

The WireGuard client in this project uses a ULA IPv6 address such as: **fd7a:115c:a1e0:92::2**

This address is valid inside the VPN network, but it cannot be routed directly on the public IPv6 Internet.

So without NAT66, the traffic would look like:

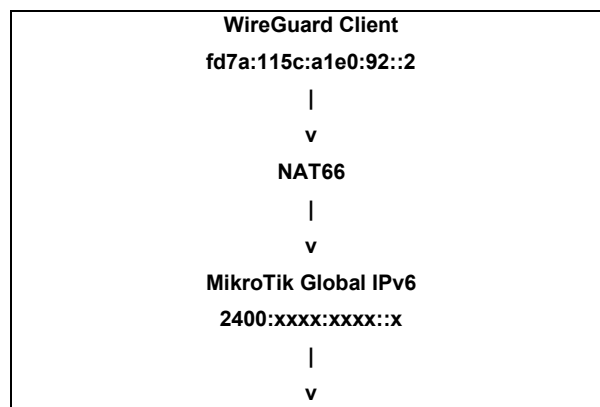
Source: **fd7a:115c:a1e0:92::2**

Destination: **2606:4700:4700::1111**

The source address is a ULA address, so it is not usable on the public Internet.

NAT66 solves this by translating the client source address to the global IPv6 address of the MikroTik router.

The process is:



IPv6 Internet

In this project, NAT66 is mainly required because the ISP-side connection provides a global IPv6 address to the MikroTik router, but the WireGuard clients use a separate ULA IPv6 subnet.

If the ISP provided a usable delegated IPv6 prefix for the clients, NAT66 might not be required.

8.2. Configure NAT66

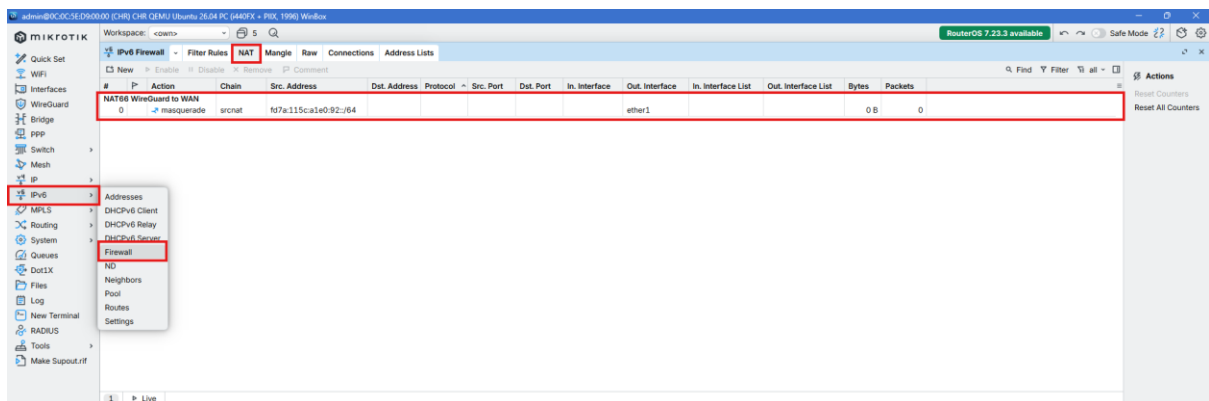
The NAT66 rule is created under the IPv6 firewall NAT section.

The source network is the WireGuard ULA network: **fd7a:115c:a1e0:92::/64** and the outgoing WAN interface is: **ether1**

CMD

```
/ipv6 firewall/nat/add chain=srcnat src-address=fd7a:115c:a1e0:92::/64 out-interface=ether1
action=masquerade comment="NAT66 WireGuard to WAN"
```

```
[admin@CHR] >
[admin@CHR] > /ipv6 firewall/nat/add chain=srcnat src-address=fd7a:115c:a1e0:92::/64 out-interface=ether1 action=masquerade comment="NAT66 WireGuard to WAN"
[admin@CHR] >
```



This rule means:

Traffic Source:
fd7a:115c:a1e0:92::/64

Going Out:
ether1

Action:
Translate source address
to MikroTik WAN IPv6

The configuration can be checked using:

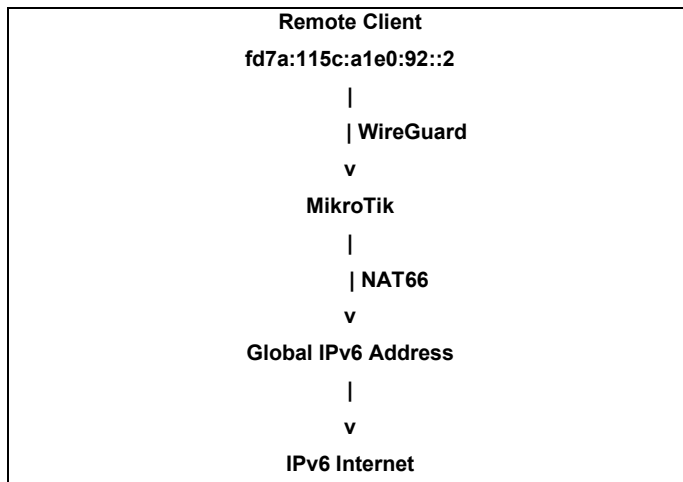
```
/ipv6 firewall nat print detail
```

```
[admin@CHR] > /ipv6 firewall nat print detail
Flags: X - DISABLED, I - INVALID; D - DYNAMIC
0      ;; NAT66 WireGuard to WAN
chain=srcnat action=masquerade src-address=fd7a:115c:a1e0:92::/64 out-interface=ether1
[admin@CHR] >
[admin@CHR] >
```

The expected rule should look similar to:

```
chain=srcnat
action=masquerade
src-address=fd7a:115c:a1e0:92::/64
out-interface=ether1
```

The complete traffic path is:



8.3. Verify NAT66 Traffic

After configuring NAT66, the rule should be tested by generating IPv6 traffic from the WireGuard client.

For example, the client can try to reach Cloudflare's IPv6 DNS server:

2606:4700:4700::1111

While the client is generating traffic, check the NAT66 counters on MikroTik:

CMD

```
/ipv6 firewall nat print stats
```

```
[admin@CHR] > /ipv6 firewall nat print stats
Columns: CHAIN, ACTION, BYTES, PACKETS
# CHAIN ACTION BYTES PACKETS
... NAT66 WireGuard to WAN
0 srcnat masquerade 0 0
[admin@CHR] > /ipv6 firewall nat print stats
Columns: CHAIN, ACTION, BYTES, PACKETS
# CHAIN ACTION BYTES PACKETS
... NAT66 WireGuard to WAN
0 srcnat masquerade 4 767 14
[admin@CHR] >
```

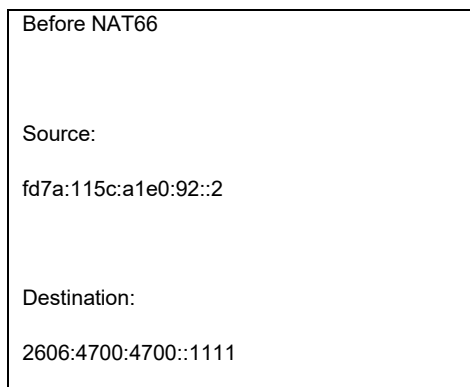
The output should show increasing values for:

BYTES

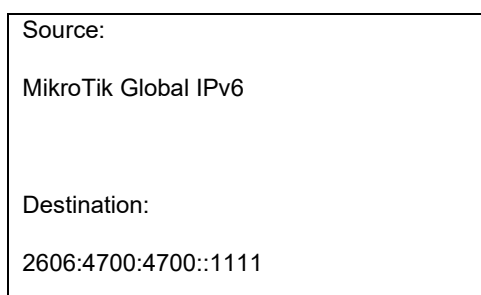
PACKETS

If the counters increase, it shows that traffic from the WireGuard client is matching the NAT66 rule.

The packet flow can be understood as:



After NAT66:



he return traffic is then translated back and sent to the WireGuard client.

Useful verification commands are:

```
/ipv6 firewall nat print detail
/ipv6 firewall nat print stats
/interface wireguard peers print detail
```

If the WireGuard handshake is active, RX/TX values are increasing, and NAT66 packet counters are also increasing, then the NAT66 configuration is working correctly.

```
[admin@CHR] >
[admin@CHR] > interface/wireguard/peers/print detail
Flags: X - DISABLED; D - DYNAMIC
0 interface=wg-vpn-ipv6 name="Remote-Client-Test-1" public-key="fl4pVVFeyrNH9Q8ctSv/WnpjedsI0NukfvJrpsYcF0" endpoint-address="" endpoint-port=0 current-endpoint-address=2408:1a08:4b61:3be9::2 current-endpoint-port=51820
  allowed-address=fd7a:115c:a1e0:92::2/128 client-address=fd7a:115c:a1e0:92::2/128 client-dns=2606:4700:4700::1111 client-endpoint=[2408:1a08:4b61:3be9::5] client-keepalive=25s client-allowed-address-:/:0 responder=yes
  responder-key=
  last-handshake=11n3s
```

Name	Interface	Public Key	Endpoint	Endpoint Port	Allowed Address	Persistent Keepalive	Current Endpoint Address	Current Endpoint Port	Rx	Tx	Last Handshake	Client
Remote-Client-Test-1	WG-VPN-IPv6	fl4pVVFeyrNH9Q8ctSv/WnpjedsI0NukfvJrpsYcF0			fd7a:115c:a1e0:92::2/128		2408:1a08:4b61:3be9::2	51820	88.8 KB	102.2 KB	00:05:01	5674

9. Testing and Verification

After completing the IPv6 WAN, WireGuard, firewall, and NAT66 configuration, the next step is to test the complete setup.

Testing is important to confirm that the WireGuard tunnel is established correctly, the client can communicate through IPv6, NAT66 is translating the traffic, and packets are passing through the expected interfaces.

The testing is performed step by step so that any problem can be identified easily.

9.1. WireGuard Handshake

The first test is to check whether the remote client has successfully established a WireGuard connection with the MikroTik router.

After enabling the WireGuard tunnel on the client, the peer information can be checked using:

CMD

```
/interface wireguard peers print detail
```

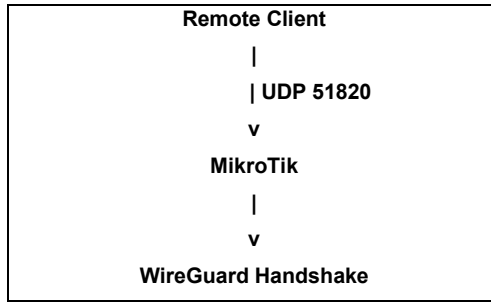
The important values to check are:

```
last-handshake
current-endpoint-address
current-endpoint-port
rx
tx
```

A successful peer may show information similar to:

Name	Interface	Public Key	Endpoint	Endpoint Port	Allowed Address	Persistent Keepalive	Current Endpoint Address	Current Endpoint Port	Rx	Tx	Last Handshake
Remote-Client-Test-1	WG-VPN-IPv6	tL4pVtEYn8HQOct5vWngJed6uSNu8hJpctcFD+	67e115ca1e092:2/128		2400:1400:4b61:3be9::2		51820		3253.7 KiB	22.5 MB	00:00:18

If **last-handshake** shows a recent time and the **rx** and **tx** values are increasing, the WireGuard tunnel is successfully established.



If the handshake does not occur, the endpoint IPv6 address, UDP port, firewall rule, keys, and client Internet connectivity should be checked.

9.2. IPv6 Ping Test

After the WireGuard handshake is successful, the next test is to verify communication inside the VPN tunnel.

The MikroTik WireGuard gateway uses: **fd7a:115c:a1e0:92::1**

The remote client uses: **fd7a:115c:a1e0:92::2**

From the Winbox, ping the Client:

```
/ping fd7a:115c:a1e0:92::2 count=5
```

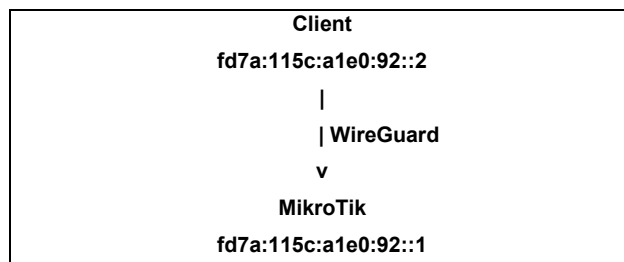
A successful reply confirms that the client can communicate with the MikroTik through the WireGuard tunnel.

```

[admin@CHR] >
[admin@CHR] > /ping fd7a:115c:a1e0:92::2 count=5
  SEQ HOST                                SIZE TTL TIME          STATUS
  ---
0 fd7a:115c:a1e0:92::2                    56  64  75ms84us  echo reply
1 fd7a:115c:a1e0:92::2                    56  64  36ms18us  echo reply
2 fd7a:115c:a1e0:92::2                    56  64  29ms654us echo reply
3 fd7a:115c:a1e0:92::2                    56  64  279ms88us echo reply
4 fd7a:115c:a1e0:92::2                    56  64  308ms100us echo reply
sent=5 received=5 packet-loss=0% min-rtt=29ms654us avg-rtt=145ms588us max-rtt=308ms100us

[admin@CHR] >
  
```

The test path is:



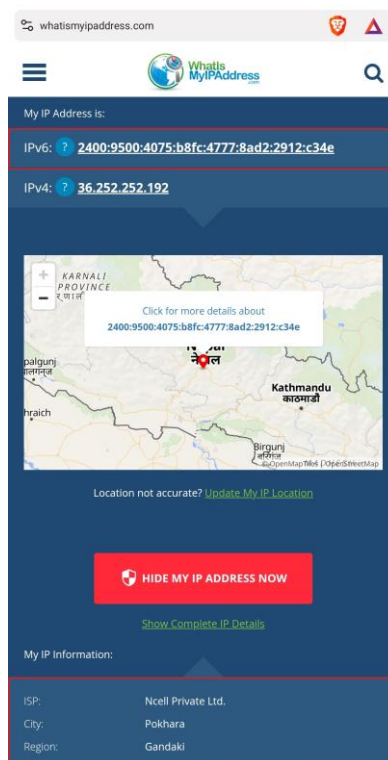
If both sides can communicate, the WireGuard IPv6 addressing and tunnel routing are working correctly.

9.3. IPv6 Internet Test

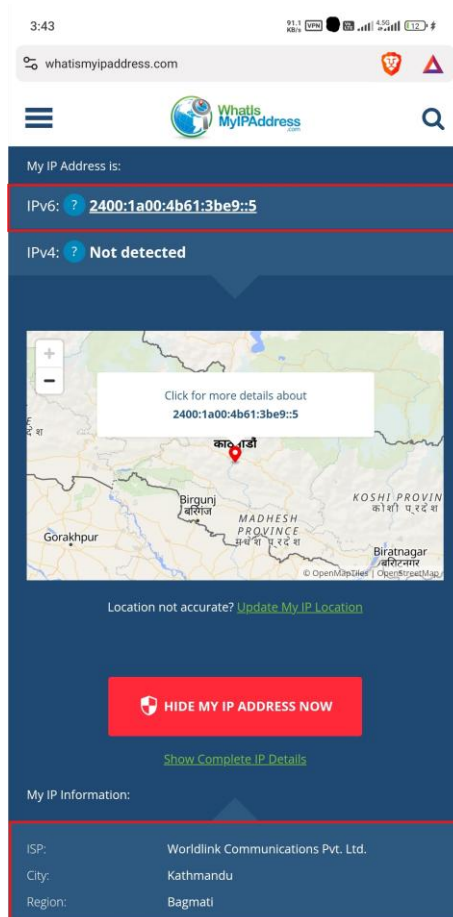
After confirming communication inside the WireGuard tunnel, the next step is to test whether the client can access the IPv6 Internet through the VPN.

For now I am going to check the IP where my IPv6 address is 2400:1a00:4b61:3be9::5

In <https://whatismyipaddress.com/> I will check whether I get winbox IP or other ISP IP

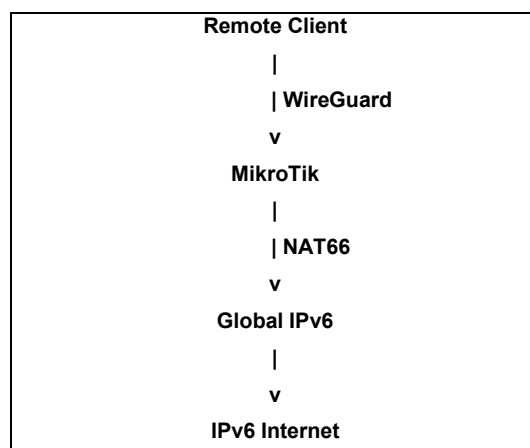


As Currently I am connected with Ncell ISP having different ipv6.



After the VPN Connection I relate to Winbox Network.

A successful result confirms that the traffic is passing through:



The MikroTik itself can also test IPv6 Internet connectivity using:
/ping 2606:4700:4700::1111 count=5

```
[admin@CHR] >
[admin@CHR] > /ping 2606:4700:4700::1111 count=5
SEQ HOST                               SIZE TTL TIME          STATUS
0 2606:4700:4700::1111                56  58 4ms720us    echo reply
1 2606:4700:4700::1111                56  58 3ms541us    echo reply
2 2606:4700:4700::1111                56  58 3ms84us     echo reply
3 2606:4700:4700::1111                56  58 4ms33us     echo reply
4 2606:4700:4700::1111                56  58 3ms715us    echo reply
sent=5 received=5 packet-loss=0% min-rtt=3ms84us avg-rtt=3ms818us max-rtt=4ms720us
[admin@CHR] >
```

9.4. NAT66 Counter Verification

After confirming IPv6 Internet access from the client, NAT66 should be verified on the MikroTik router.

The NAT66 statistics can be checked using:

CMD

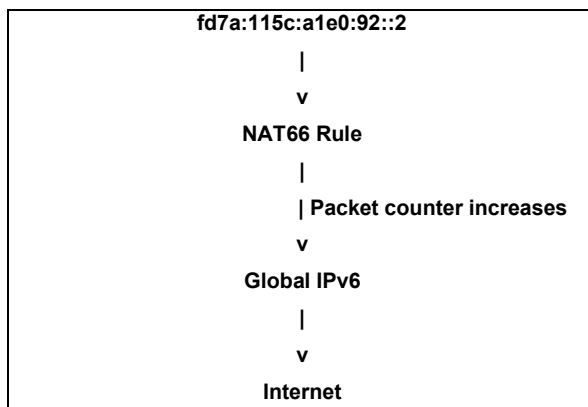
```
/ipv6 firewall nat print stats
```

The output should show the NAT66 rule with packet and byte counters.

```
[admin@CHR] > /ipv6 firewall nat print stats
Columns: CHAIN, ACTION, BYTES, PACKETS
# CHAIN  ACTION      BYTES  PACKETS
;;; NAT66 WireGuard to WAN
0 srcnat  masquerade  517 881    1 330
[admin@CHR] >
```

If the BYTES and PACKETS values continue increasing, it confirms that the WireGuard client traffic is passing through the NAT66 rule.

The process is:



This provides clear evidence that NAT66 is actively translating the VPN client's IPv6 traffic.

9.5. Packet and Traffic Analysis

The final test is to observe the actual traffic passing through the MikroTik router. RouterOS provides tools such as Torch and Packet Sniffer that can be used to monitor IPv6 traffic.

Using Torch

Traffic on the WireGuard interface can be observed using:

CMD

```
/tool torch WG-VPN-IPv6
```

```
/tool torch ether1
```

This can show traffic entering and leaving the VPN interface.

```
[admin@CHR] > /tool torch WG-VPN-IPv6
Columns: MAC-PROTOCOL, IP-PROTOCOL, DSCP, SRC-ADDRESS, SRC-PORT, DST-ADDRESS, DST-PORT, TX, RX, TX-PACKETS, RX-PACKETS
MAC-PROTOCOL  IP-PROTOCOL  DSCP  SRC-ADDRESS  SRC-PORT  DST-ADDRESS  DST-PORT  TX  RX  TX-PACKETS  RX-PACKETS
ipv6          tcp          0      fd7a:115c:a1e0:92::2  48872     2606:4700:4700::1111  853      0bps 0bps      0      0
ipv6          tcp          0      fd7a:115c:a1e0:92::2  49928     2803:f800:53::3      443 (https) 0bps 0bps      0      0
ipv6          tcp          0      fd7a:115c:a1e0:92::2  39142     2803:f800:53::7      443 (https) 0bps 0bps      0      0
ipv6          tcp          0      fd7a:115c:a1e0:92::2  46246     2a03:2800:f312:121::face:b00c:0:7260  5222     576bps 576bps    1      1
ipv6          tcp          34     fd7a:115c:a1e0:92::2  48872     2606:4700:4700::1111  853      0bps 0bps      0      0
ipv6          tcp          34     fd7a:115c:a1e0:92::2  49928     2803:f800:53::3      443 (https) 0bps 0bps      0      0
ipv6          tcp          34     fd7a:115c:a1e0:92::2  39142     2803:f800:53::7      443 (https) 0bps 0bps      0      0
ipv6          udp          0      fd7a:115c:a1e0:92::2  44418     2606:4700::6813:de4f  443 (https) 0bps 0bps      0      0
ipv6          udp          0      fd7a:115c:a1e0:92::2  44419     2803:f800:53::3      443 (https) 0bps 0bps      0      0
ipv6          udp          0      fd7a:115c:a1e0:92::2  37856     2803:f800:53::7      443 (https) 0bps 0bps      0      0
ipv6          udp          10     fd7a:115c:a1e0:92::2  44418     2606:4700::6813:de4f  443 (https) 0bps 0bps      0      0
ipv6          udp          10     fd7a:115c:a1e0:92::2  44419     2803:f800:53::3      443 (https) 0bps 0bps      0      0
ipv6          udp          10     fd7a:115c:a1e0:92::2  37856     2803:f800:53::7      443 (https) 0bps 0bps      0      0
```

Using Packet Sniffer

The MikroTik packet sniffer can also be used:

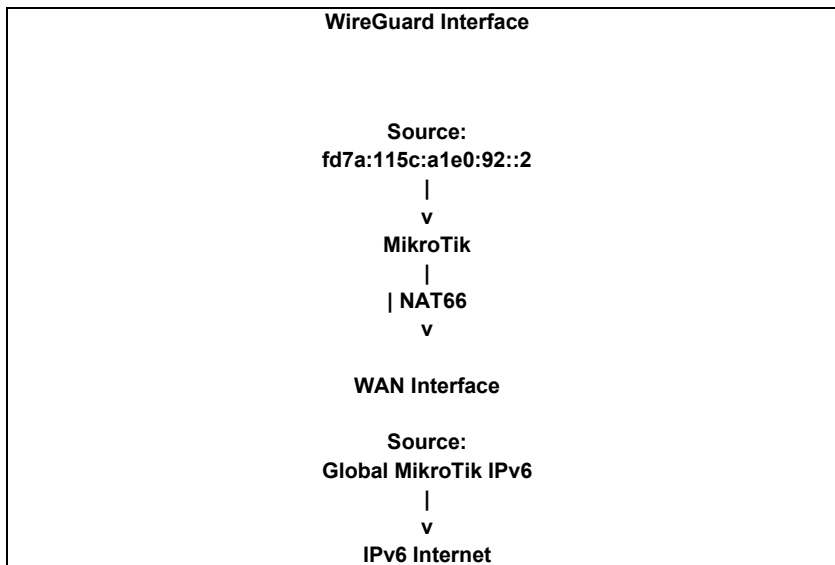
CMD

```
/tool sniffer quick interface=WG-VPN-IPv6
```

```
/tool sniffer quick interface=ether1
```

```
[admin@CHR] > /tool sniffer quick interface=WG-VPN-IPv6
Columns: INTERFACE, TIME, NUM, DIR, SRC-ADDRESS, DST-ADDRESS, PROTOCOL, SIZE, CPU
INTERFACE  TIME    NUM  DIR  SRC-ADDRESS          DST-ADDRESS          PROTOCOL  SIZE  CPU
WG-VPN-IPv6 3.689   57   →   2606:4700::6813:de4f:443 (https) fd7a:115c:a1e0:92::2:40656  ipv6:udp 1248  0
WG-VPN-IPv6 3.689   58   →   2606:4700::6813:de4f:443 (https) fd7a:115c:a1e0:92::2:40656  ipv6:udp 1248  0
WG-VPN-IPv6 3.689   59   →   2606:4700::6813:de4f:443 (https) fd7a:115c:a1e0:92::2:40656  ipv6:udp 1248  0
WG-VPN-IPv6 3.689   60   →   2606:4700::6813:de4f:443 (https) fd7a:115c:a1e0:92::2:40656  ipv6:udp 1248  0
WG-VPN-IPv6 3.689   61   →   2606:4700::6813:de4f:443 (https) fd7a:115c:a1e0:92::2:40656  ipv6:udp 1248  0
WG-VPN-IPv6 3.689   62   →   2606:4700::6813:de4f:443 (https) fd7a:115c:a1e0:92::2:40656  ipv6:udp 760   0
WG-VPN-IPv6 3.821   63   ←   fd7a:115c:a1e0:92::2:40656  2606:4700::6813:de4f:443 (https)  ipv6:udp 93    0
WG-VPN-IPv6 3.826   64   ←   fd7a:115c:a1e0:92::2:40656  2606:4700::6813:de4f:443 (https)  ipv6:udp 94    0
WG-VPN-IPv6 3.836   65   ←   fd7a:115c:a1e0:92::2:40656  2606:4700::6813:de4f:443 (https)  ipv6:udp 92    0
WG-VPN-IPv6 3.841   66   ←   fd7a:115c:a1e0:92::2:40656  2606:4700::6813:de4f:443 (https)  ipv6:udp 93    0
WG-VPN-IPv6 3.845   67   ←   fd7a:115c:a1e0:92::2:40656  2606:4700::6813:de4f:443 (https)  ipv6:udp 92    0
WG-VPN-IPv6 3.845   68   ←   fd7a:115c:a1e0:92::2:40656  2606:4700::6813:de4f:443 (https)  ipv6:udp 94    0
WG-VPN-IPv6 3.845   69   ←   fd7a:115c:a1e0:92::2:40656  2606:4700::6813:de4f:443 (https)  ipv6:udp 93    0
WG-VPN-IPv6 3.845   70   ←   fd7a:115c:a1e0:92::2:40656  2606:4700::6813:de4f:443 (https)  ipv6:udp 93    0
WG-VPN-IPv6 3.845   71   ←   fd7a:115c:a1e0:92::2:40656  2606:4700::6813:de4f:443 (https)  ipv6:udp 91    0
WG-VPN-IPv6 3.845   72   ←   fd7a:115c:a1e0:92::2:40656  2606:4700::6813:de4f:443 (https)  ipv6:udp 93    0
WG-VPN-IPv6 3.845   73   ←   fd7a:115c:a1e0:92::2:40656  2606:4700::6813:de4f:443 (https)  ipv6:udp 91    0
WG-VPN-IPv6 3.845   74   ←   fd7a:115c:a1e0:92::2:40656  2606:4700::6813:de4f:443 (https)  ipv6:udp 93    0
WG-VPN-IPv6 3.846   75   ←   fd7a:115c:a1e0:92::2:40656  2606:4700::6813:de4f:443 (https)  ipv6:udp 91    0
WG-VPN-IPv6 3.846   76   ←   fd7a:115c:a1e0:92::2:40656  2606:4700::6813:de4f:443 (https)  ipv6:udp 91    0
[Q quit]D dump[C-z pause]
```

The expected traffic path is:



This analysis helps demonstrate the difference between the traffic before NAT66 and after NAT66.

For more detailed packet-level analysis, traffic can also be captured and examined using Wireshark in the GNS3 environment.

At the end of the testing process, the following should be successfully verified:

Test	Expected Result
WireGuard Handshake	Established

WireGuard RX/TX	Increasing
Client Ping	Successful
IPv6 Internet Check	Successful
Nat66 Packets	Increasing
NAT66 Bytes	Increasing
WireGuard Traffic	Visible
WAN IPv6 Traffic	Visible

If all of these tests are successful, the IPv6-only WireGuard VPN with NAT66 is working correctly.

10. Troubleshooting

During the implementation, a few issues may appear while configuring WireGuard, IPv6 routing, QR-code client import, and NAT66. Troubleshooting should be done step by step so that the exact point of failure can be identified.

The main areas to check are the WireGuard endpoint, client configuration, IPv6 routing, firewall rules, and NAT66 translation.

10.1. WireGuard Endpoint Issues

One of the common problems is an incorrect WireGuard endpoint.

For IPv6, the endpoint must use square brackets around the IPv6 address.

Correct format: **[2400:xxxx:xxxx::5]:51820**

Incorrect formats:

2400:xxxx:xxxx::5:51820

or:

[2400:xxxx:xxxx::5]:51820:51820

If the endpoint is incorrect, the client may fail to connect or the QR code may generate an invalid configuration.

The endpoint can be checked using:

```
/interface wireguard peers print detail
```

The client configuration should show: Endpoint = [GLOBAL-IPv6-ADDRESS]:51820

Also check that the WireGuard interface is listening on the correct port:

```
/interface wireguard print detail
```

Expected: listen-port=51820

If the endpoint is correct but there is still no connection, verify that the IPv6 firewall allows UDP port 51820.

10.2. QR Code Import Issues

The QR code can fail to import if the generated WireGuard configuration contains an invalid endpoint or incomplete client settings.

The most common issue is incorrect IPv6 endpoint formatting.

The correct generated configuration should look similar to:

```
[Interface]
Address = fd7a:115c:a1e0:92::2/128
DNS = 2606:4700:4700::1111

[Peer]
AllowedIPs = ::/0
Endpoint = [2400:xxxx:xxxx::5]:51820
PersistentKeepalive = 25
```

If the QR code gives an error such as: **Cannot parse endpoint** check the **Endpoint** line first.

The client configuration can be generated again using:

```
/interface wireguard peers show-client-config Remote-Client
```

If the peer configuration is wrong, it is often easier to remove the peer and create it again correctly.

```
/interface wireguard peers remove [find name="Remote-Client"]
```

Then recreate the peer with the correct client settings.

Private keys and QR codes should not be shared publicly because they contain sensitive VPN credentials.

10.3. IPv6 Routing Issues

Sometimes the WireGuard tunnel may establish successfully, but the client still cannot access the IPv6 Internet.

In this case, the IPv6 routing table should be checked.

Use:

```
/ipv6 route print detail
```

The router should have an active default route:

```
::/0
```

It should also have a connected route for the WireGuard ULA network:

```
fd7a:115c:a1e0:92::/64
```

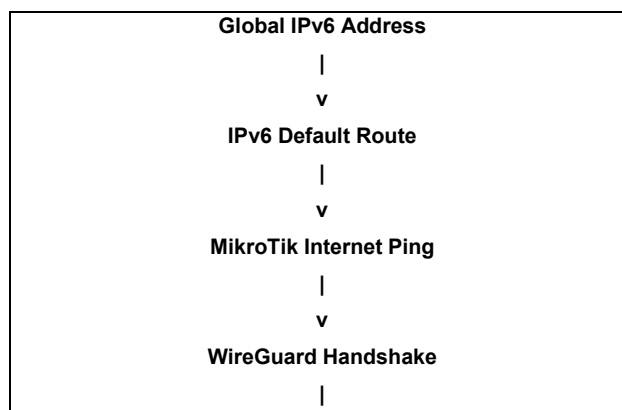
The MikroTik should first be able to reach the IPv6 Internet directly.

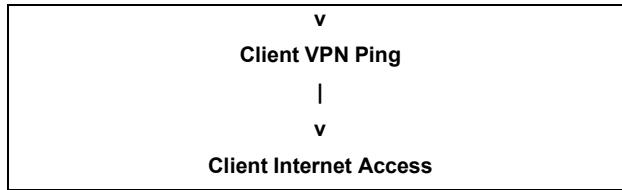
Test:

```
/ping 2606:4700:4700::1111 count=5
```

If the MikroTik itself cannot reach the IPv6 Internet, the problem is not WireGuard. The WAN IPv6 address, default route, or ISP-side connectivity should be checked first.

The troubleshooting order should be:





This makes it easier to identify where the traffic is failing.

10.4. NAT66 Issues

Another common problem is that the WireGuard handshake works and the client can ping the MikroTik VPN gateway, but the client cannot access the IPv6 Internet. In this situation, NAT66 should be checked.

First, verify the NAT66 rule:

```
/ipv6 firewall nat print detail
```

The rule should contain:

```
chain=srcnat
src-address=fd7a:115c:a1e0:92::/64
out-interface=ether1
action=masquerade
```

Then check the NAT66 counters:

```
/ipv6 firewall nat print stats
```

Generate traffic from the WireGuard client and check whether the packet and byte counters increase.

If the counters remain at zero, the client traffic is not matching the NAT66 rule.

Check:

- Correct source ULA network
- Correct WAN interface
- WireGuard client address
- IPv6 forwarding firewall rule
- Client AllowedIPs
- IPv6 default route

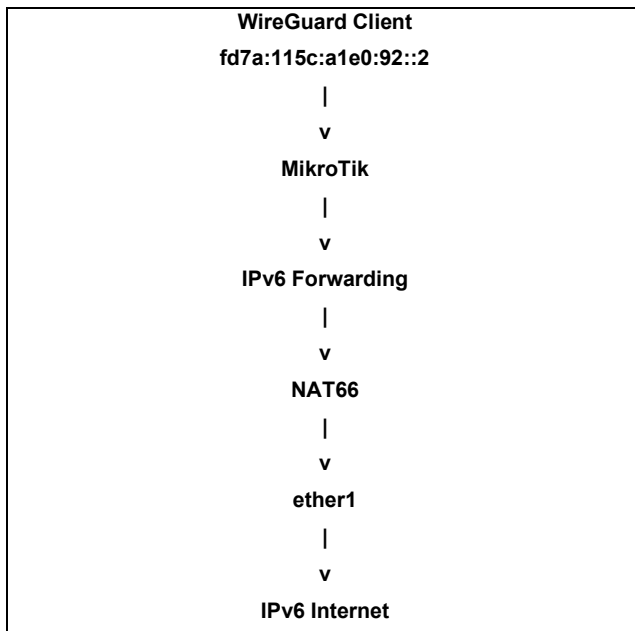
For example, the client should use:

```
fd7a:115c:a1e0:92::2/128
```

And

```
AllowedIPs = ::/0
```

The expected traffic flow is:



If the WireGuard handshake is active, the VPN gateway is reachable, and NAT66 counters are increasing, then the main WireGuard and NAT66 configuration is working correctly.

A useful troubleshooting command set is:

```

/interface wireguard print detail
/interface wireguard peers print detail
/ipv6 address print detail
/ipv6 route print detail
/ipv6 firewall filter print detail
/ipv6 firewall nat print detail
/ipv6 firewall nat print stats
  
```

11. Cost and Real-World Use for SMEs

The main purpose of this project is not only to configure WireGuard and NAT66, but also to show how IPv6 can be useful for small and medium-sized businesses that need remote access without depending on an expensive public IPv4 connection.

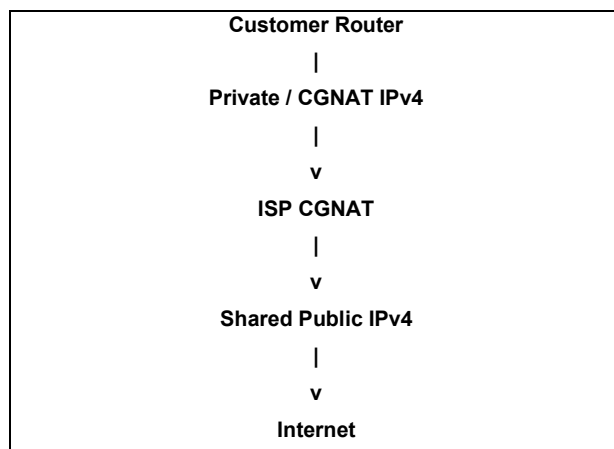
Many SMEs already use normal broadband Internet connections. If their ISP provides usable IPv6 connectivity, the same Internet connection can potentially be used to host an IPv6-based WireGuard VPN.

11.1. Public IPv4 vs IPv6

With IPv4, many ISPs place customers behind CGNAT because public IPv4 addresses are limited.

In this situation, the customer may receive an address such as: 100.64.x.x instead of a directly reachable public IPv4 address.

The traffic flow is:



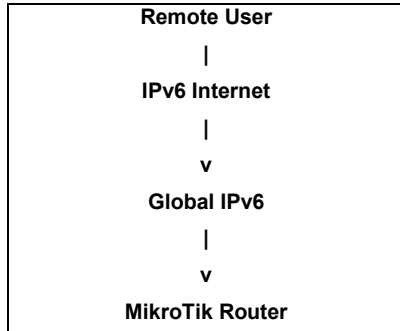
This normally works for Internet browsing, but incoming connections are more difficult because the public IPv4 address is controlled by the ISP.

For services such as WireGuard, the customer may need:

- A public IPv4 address from the ISP
- A static IPv4 service
- A business Internet package
- A dedicated Internet connection

IPv6 provides a different approach.

An ISP can provide a globally routable IPv6 address directly to the customer's router.



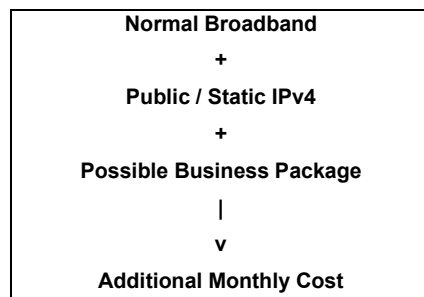
This global IPv6 address can be used as the WireGuard endpoint without depending on a public IPv4 address.

However, IPv6 availability and inbound connectivity still depend on the ISP and firewall configuration.

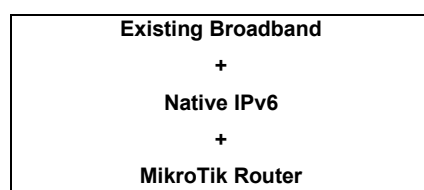
11.2. Cost Benefits

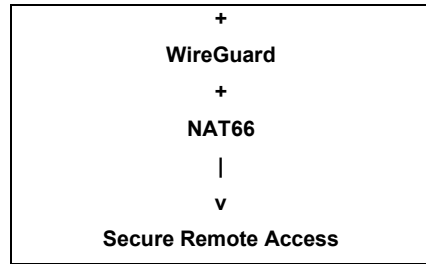
One of the main benefits of this project is that it can reduce the need to purchase additional Internet services only for remote VPN access.

A traditional approach may look like:



The proposed solution uses:





WireGuard itself is free to use, and it is already supported by MikroTik RouterOS.

Therefore, if the business already has:

- An IPv6-capable Internet connection
- A compatible MikroTik router
- Basic networking knowledge

the additional cost of creating the VPN can be very low.

This can be useful for small businesses that only need secure access to office resources such as:

- Internal servers
- Monitoring systems
- File services
- Router management
- Web applications
- Other authorized office resources

The solution does not mean that dedicated Internet connections are unnecessary in every situation. Businesses that require guaranteed bandwidth, static addressing, high availability, SLA support, or enterprise-level redundancy may still need dedicated services.

11.3. Real-World Deployment Considerations

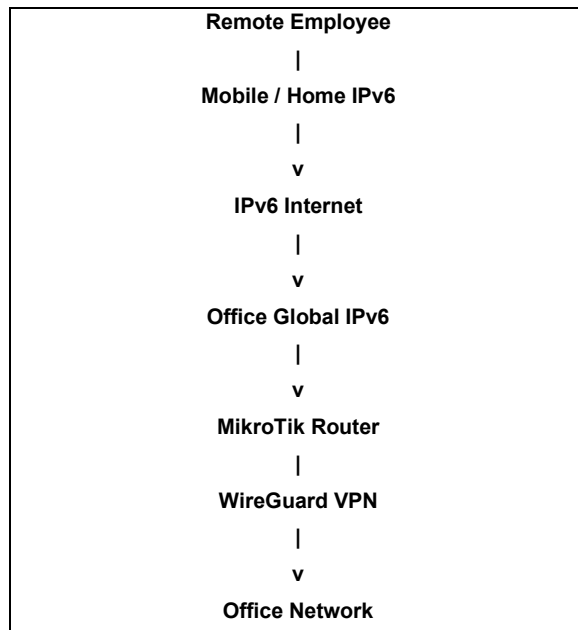
Although the project is implemented and tested in GNS3, several factors should be considered before using the same design in a real business environment.

The first requirement is that the ISP must provide working IPv6 connectivity.

The MikroTik router should receive a globally routable IPv6 address that can be reached from the remote client's network.

The ISP should also allow incoming IPv6 traffic to the WireGuard UDP port.

The expected real-world design would be:



Other important considerations include:

- The remote user's Internet connection should support IPv6.
- IPv6 firewall rules should be properly configured.
- Only required WireGuard traffic should be allowed from the Internet.
- WireGuard private keys must be protected.
- Different users should have separate WireGuard peers.
- Router management access should be restricted.
- Configuration backups should be maintained.
- Logs and VPN connections should be monitored.
- Dynamic IPv6 address changes should be considered.
- If the ISP provides a delegated IPv6 prefix, routing may be preferred instead of NAT66.

For a small business, one compatible MikroTik router may be enough to provide both normal network routing and WireGuard remote access.

The proposed architecture is therefore suitable mainly for SMEs that already have IPv6 connectivity and need a simple, secure, and low-cost remote access solution without purchasing a public IPv4 service only for VPN connectivity.

The GNS3 implementation in this project demonstrates that the design is technically possible and can later be adapted to physical MikroTik RouterOS hardware for real-world use.

12. Results and Conclusion

The IPv6-only WireGuard VPN with NAT66 was successfully configured and tested using MikroTik RouterOS in the GNS3 environment.

During the implementation, the MikroTik router was first provided with working IPv6 WAN connectivity. After confirming the global IPv6 address and IPv6 default route, a WireGuard interface was created and a separate ULA IPv6 network was assigned to the VPN.

The WireGuard client was able to establish a successful handshake with the MikroTik router. Communication between the client and the WireGuard gateway was also verified using IPv6 ping tests.

NAT66 was then configured to translate traffic from the WireGuard ULA network to the global IPv6 address available on the WAN interface. The NAT66 packet and byte counters increased when the client generated Internet traffic, confirming that the translation was working correctly.

The main results of the project are summarized below:

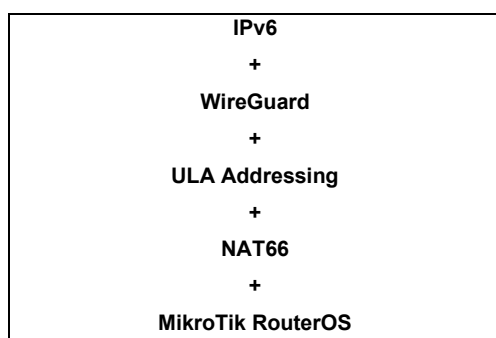
Test	Result
IPv6 WAN Connectivity	Successful
Global IPv6 Address	Successfully Assigned
IPv6 Default Route	Active
WireGuard Interface	Running

WireGuard Handshake	Successful
VPN IPv6 Communication	Successful
IPv6 Internet Access	Successful
NAT66 Translation	Successful
NAT66 Counters	Increasing
IPv6 Firewall Rules	Working

The project demonstrated that IPv6 can be used as an alternative to depending on a public IPv4 address for establishing a remote-access VPN.

One of the main problems with IPv4 is that many ISP customers are placed behind CGNAT, which makes direct incoming connections difficult. By using a globally reachable IPv6 address as the WireGuard endpoint, this limitation can be avoided when both the office and remote network have working IPv6 connectivity.

The final solution uses:



to provide secure remote connectivity.

The implementation also shows that this approach can be useful for small and medium-sized businesses that already have IPv6 connectivity and need remote access without purchasing a public IPv4 service only for VPN purposes.

This project was implemented and tested in GNS3 using MikroTik RouterOS. A physical MikroTik deployment was not performed as part of this project. However, the successful GNS3 implementation demonstrates that the same design can be considered for future deployment on compatible physical MikroTik hardware.

13. References

The following references can be used for the concepts and technologies discussed in this report.

MikroTik. WireGuard – RouterOS Documentation. MikroTik Documentation.
 MikroTik. IPv6 – RouterOS Documentation. MikroTik Documentation.
 MikroTik. IPv6 Firewall – RouterOS Documentation. MikroTik Documentation.
 MikroTik. DHCPv6 Client – RouterOS Documentation. MikroTik Documentation.
 WireGuard. WireGuard: Fast, Modern, Secure VPN Tunnel. WireGuard Project.
 IETF. RFC 8200 – Internet Protocol, Version 6 (IPv6) Specification.
 IETF. RFC 4193 – Unique Local IPv6 Unicast Addresses.
 IETF. RFC 6598 – IANA-Reserved IPv4 Prefix for Shared Address Space.
 GNS3. GNS3 Documentation. Graphical Network Simulator Documentation.
 MikroTik. Cloud Hosted Router (CHR) Documentation. MikroTik Documentation.

14. Appendix

The appendix contains the main MikroTik configuration and verification commands used during the project. Sensitive information such as WireGuard private keys should not be included in the public version of the report.

14.1. Complete MikroTik Configuration

The following configuration represents the main setup used for the IPv6-only WireGuard VPN with NAT66.

```
/interface ethernet
set [ find default-name=ether1 ] disable-running-check=no
set [ find default-name=ether2 ] disable-running-check=no
set [ find default-name=ether3 ] disable-running-check=no
set [ find default-name=ether4 ] disable-running-check=no
set [ find default-name=ether5 ] disable-running-check=no
set [ find default-name=ether6 ] disable-running-check=no
set [ find default-name=ether7 ] disable-running-check=no
set [ find default-name=ether8 ] disable-running-check=no
set [ find default-name=ether9 ] disable-running-check=no
set [ find default-name=ether10 ] disable-running-check=no
set [ find default-name=ether11 ] disable-running-check=no
```

```

set [ find default-name=ether12 ] disable-running-check=no
set [ find default-name=ether13 ] disable-running-check=no
set [ find default-name=ether14 ] disable-running-check=no
set [ find default-name=ether15 ] disable-running-check=no
set [ find default-name=ether16 ] disable-running-check=no
set [ find default-name=ether17 ] disable-running-check=no
set [ find default-name=ether18 ] disable-running-check=no
set [ find default-name=ether19 ] disable-running-check=no
set [ find default-name=ether20 ] disable-running-check=no
set [ find default-name=ether21 ] disable-running-check=no
set [ find default-name=ether22 ] disable-running-check=no
set [ find default-name=ether23 ] disable-running-check=no
set [ find default-name=ether24 ] disable-running-check=no
set [ find default-name=ether25 ] disable-running-check=no
set [ find default-name=ether26 ] disable-running-check=no
set [ find default-name=ether27 ] disable-running-check=no
set [ find default-name=ether28 ] disable-running-check=no
set [ find default-name=ether29 ] disable-running-check=no
set [ find default-name=ether30 ] disable-running-check=no
set [ find default-name=ether31 ] disable-running-check=no
set [ find default-name=ether32 ] disable-running-check=no

/interface wireguard
add comment="IPv6 WireGuard VPN" listen-port=51820 mtu=1420 name=WG-VPN-IPv6 private-key=\
    "0ezO/YBHiqq+opp3ulOkwQJDP1fq5JYSFSOfAk9POo="

/interface wireguard peers
add allowed-address=fd7a:115c:a1e0:92::2/128 client-address=fd7a:115c:a1e0:92::2/128 client-allowed-address=::/0 \
    client-dns=2606:4700:4700::1111 client-endpoint="[2400:1a00:4b61:3be9::5]" client-keepalive=25s interface=\
    WG-VPN-IPv6 name=Remote-Client-Test-1 private-key="wCS+DEBm5bB8o7WDgWCK3727J12MCIAZ2S953DdnoFc="
public-key=\
    "fL4pVVfEyrNH9Q0ctSv/Wnpjed6sI0NukfvJrpsYcF0=" responder=yes

/ip dhcp-client
add interface=ether1 name=client1

/ipv6 address
add address=fd7a:115c:a1e0:92::1 advertise=no comment="WireGuard IPv6 Gateway" interface=WG-VPN-IPv6

/ipv6 dhcp-client
add add-default-route=yes comment="DHCP-CLINET REQUEST TO ISP ROUTER" interface=ether1 request=address \
    use-interface-duid=yes

/ipv6 firewall filter
add action=accept chain=input comment="Allow WireGuard IPv6" dst-port=51820 in-interface=ether1 protocol=udp
add action=accept chain=input comment="Allow WG clients to router" in-interface=WG-VPN-IPv6 src-address=\
    fd7a:115c:a1e0:92::/64
add action=accept chain=forward comment="Allow WG IPv6 forwarding" in-interface=WG-VPN-IPv6 src-address=\
    fd7a:115c:a1e0:92::/64

/ipv6 firewall nat

```

```
add action=masquerade chain=srcnat comment="NAT66 WireGuard to WAN" out-interface=ether1 src-address=\
fd7a:115c:a1e0:92::/64
```

14.2. Verification Command

The following commands were used to verify each part of the implementation.

Check DHCPv6 Client

```
/ipv6 dhcp-client print detail
```

Check IPv6 Addresses

```
/ipv6 address print detail
```

Check IPv6 Routing Table

```
/ipv6 route print detail
```

The router should have an active: **::/0** default route.

Test Router IPv6 Internet

```
/ping 2606:4700:4700::1111 count=5
```

During the validated test, all five packets were successfully received with 0% packet loss.

Check WireGuard Interface

```
/interface wireguard print detail
```

Check WireGuard Peer

```
/interface wireguard peers print detail
```

Check:

```
last-handshake
current-endpoint-address
current-endpoint-port
rx
tx
```

The working peer showed active RX/TX traffic and a successful handshake.

Test WireGuard Gateway

From the client:

```
ping fd7a:115c:a1e0:92::1
```

From MikroTik:

```
/ping fd7a:115c:a1e0:92::2 count=5
```

Check IPv6 Firewall

```
/ipv6 firewall filter print detail
```

Check NAT66 Configuration

```
/ipv6 firewall nat print detail
```

In the validated test, the NAT66 rule had already translated **2,750** packets, confirming that client traffic was passing through the rule.

Monitor WireGuard Traffic

```
/tool torch WG-VPN-IPv6
```

Monitor WAN IPv6 Traffic

```
/tool torch ether1
```

Packet Sniffer

```
/tool sniffer quick interface=WG-VPN-IPv6
```

or:

```
/tool sniffer quick interface=ether1
```

These verification commands provide evidence that the complete path is working:

Remote Client

|

